

Manual Técnico

Instituto Tecnológico de Durango



Departamento: Ingeniería en Sistemas Computacionales

LENGUAJES Y AUTOMATAS II | 7y

Manual Técnico

Romo Bernal Axell Roy 22041195

Elda Rivera Saucedo

Victoria de Durango Dgo, 29 de Noviembre de 2025

INTRODUCCIÓN

Este documento presenta el manual técnico del compilador **Talos**, el objetivo principal de este compilador es analizar, validar y procesar un lenguaje de programación, realiza acciones como detectar errores lógicos y de coherencia en el código fuente antes de que este sea ejecutado o traducido a lenguaje de bajo nivel.

A lo largo del documento se detalla en diferentes secciones por el cual se conforma el analizador, así como la lógica que tiene detrás, como secciones de código y diagramas con acciones y el cómo se llegó a las matrices realizadas en Excel para posteriormente integrarla en código. Las secciones en las que se desglosa son:

Analizador Léxico: La primera fase, responsable de escanear el código fuente. Se detalla el Autómata Finito Determinista (AFD) implementado a través de una matriz de transiciones, el cual convierte el flujo de caracteres en una secuencia de tokens.

Analizador Sintáctico: La segunda fase, que verifica la estructura gramatical del código. Se explica la implementación de una matriz predictiva, detallando la gramática libre de contexto utilizada, la construcción de la Matriz Predictiva y el algoritmo de análisis basado en pila que valida que la secuencia de tokens cumpla con las reglas sintácticas del lenguaje.

Analizador Semántico: Responsable de verificar el significado y la coherencia lógica del programa. Esta sección explica en profundidad la implementación de la **Tabla de Símbolos** para la gestión de variables, el uso de **pilas semánticas** para la evaluación de expresiones, y la implementación de un Cubo Semántico para la validación de tipos. Se documentan las acciones semánticas insertadas en la gramática para detectar errores como variables no declaradas, duplicidad de identificadores y tipos incompatibles en operaciones.

ANALIZADOR LÉXICO

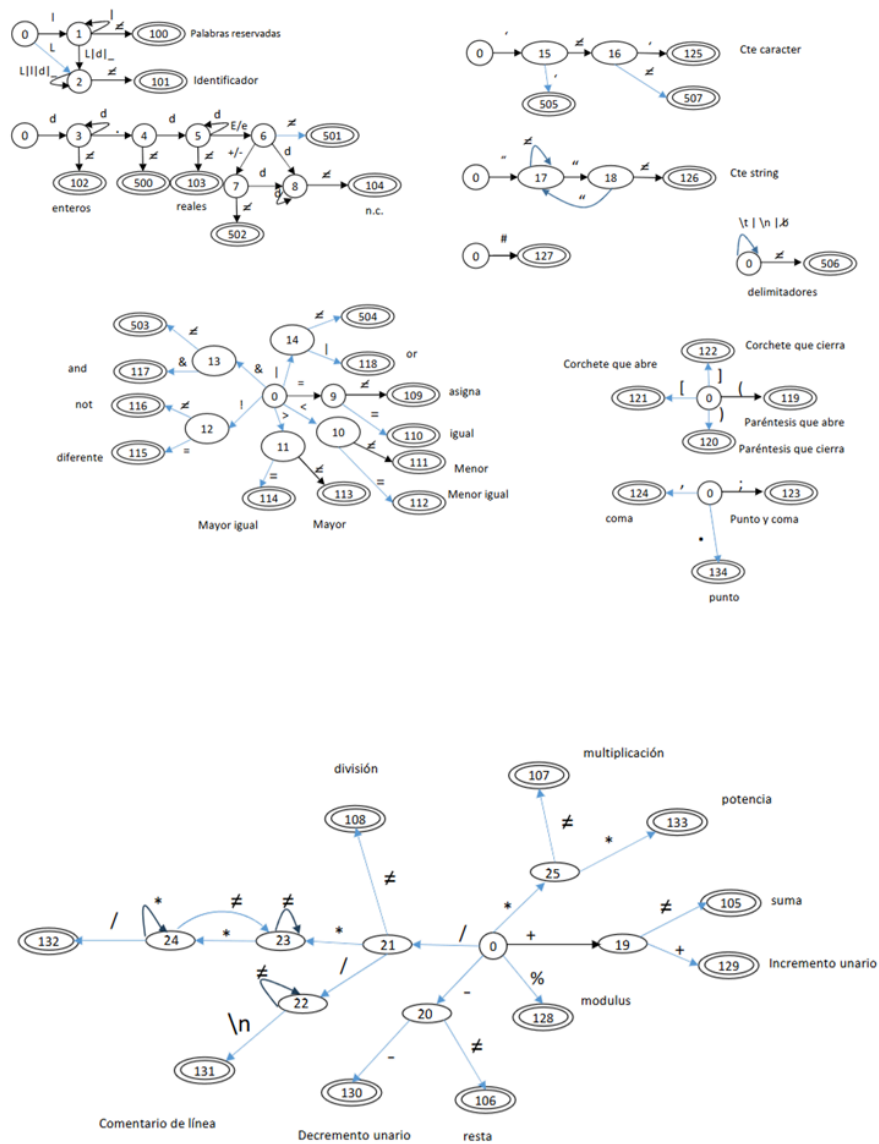
Talos: Autómata para el Analizador Léxico

Palabras Reservadas

- include
- lib
- endlib
- class
- endclass
- int
- float
- char
- string
- bool
- void
- if
- elseif
- else
- endif
- do
- enddo
- while
- endwhile
- read
- write
- def
- const
- of
- dowhile
- for
- endfor
- function
- endfunction
- return

Conjuntos

- l = letras minúsculas
- L = letras mayúsculas
- d = dígitos 0 al 9



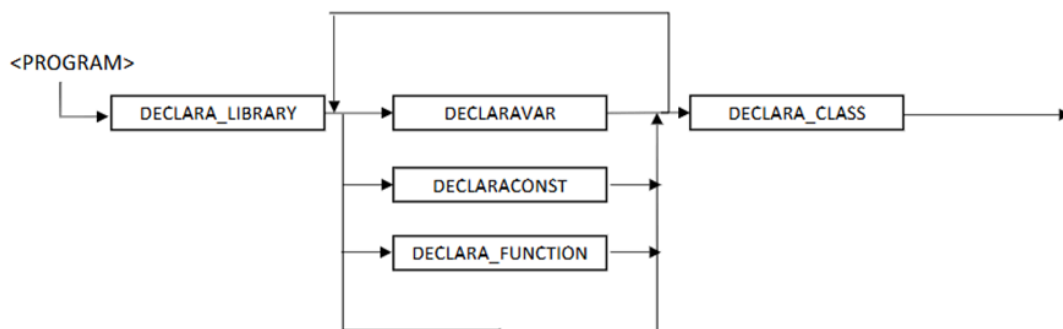
Matriz de Transición

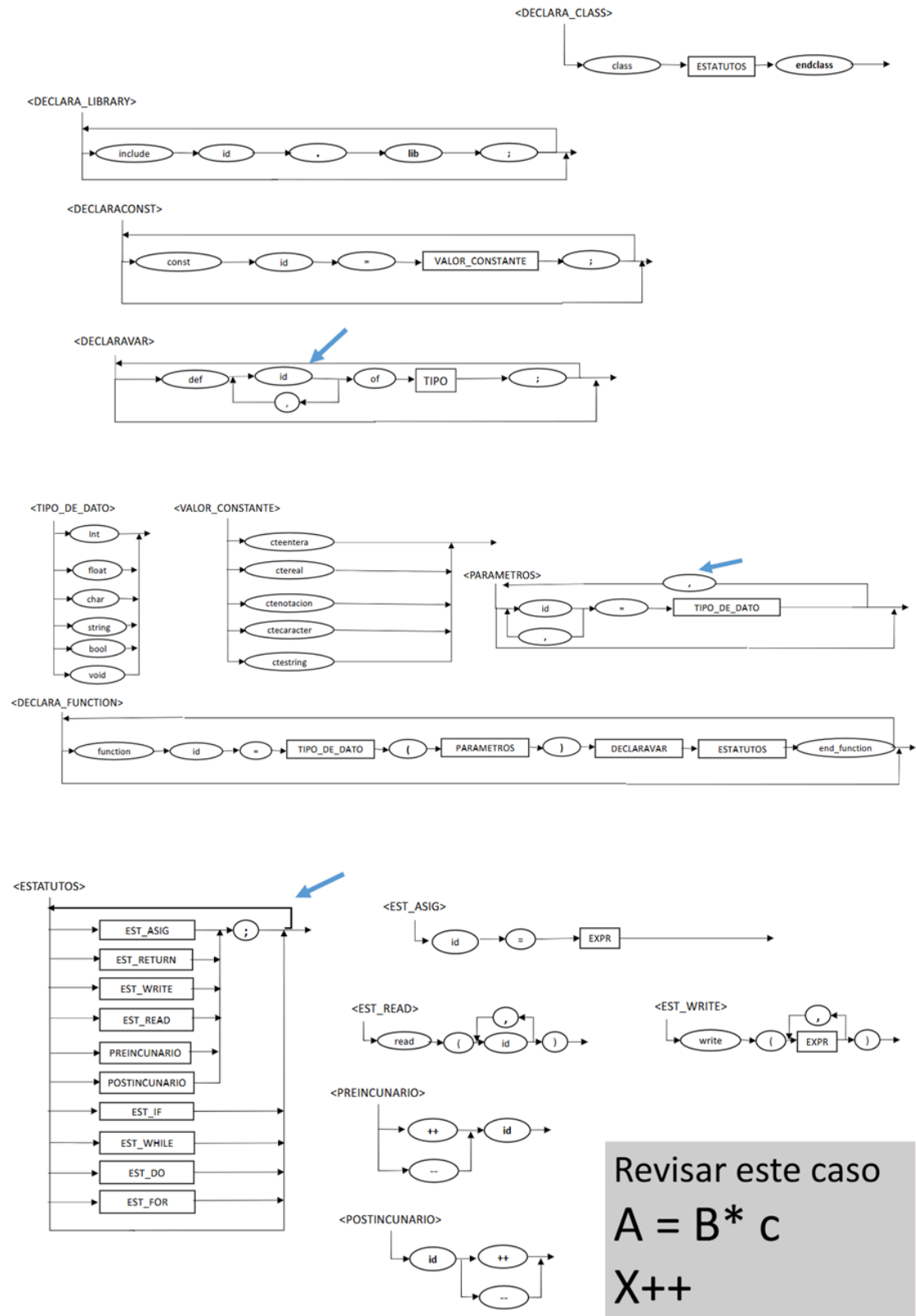
Hecha a base con los diagramas anteriormente vistos, donde su función es establecer la relación entre cada carácter de entrada y el token correspondiente, de acuerdo con las reglas del analizador léxico

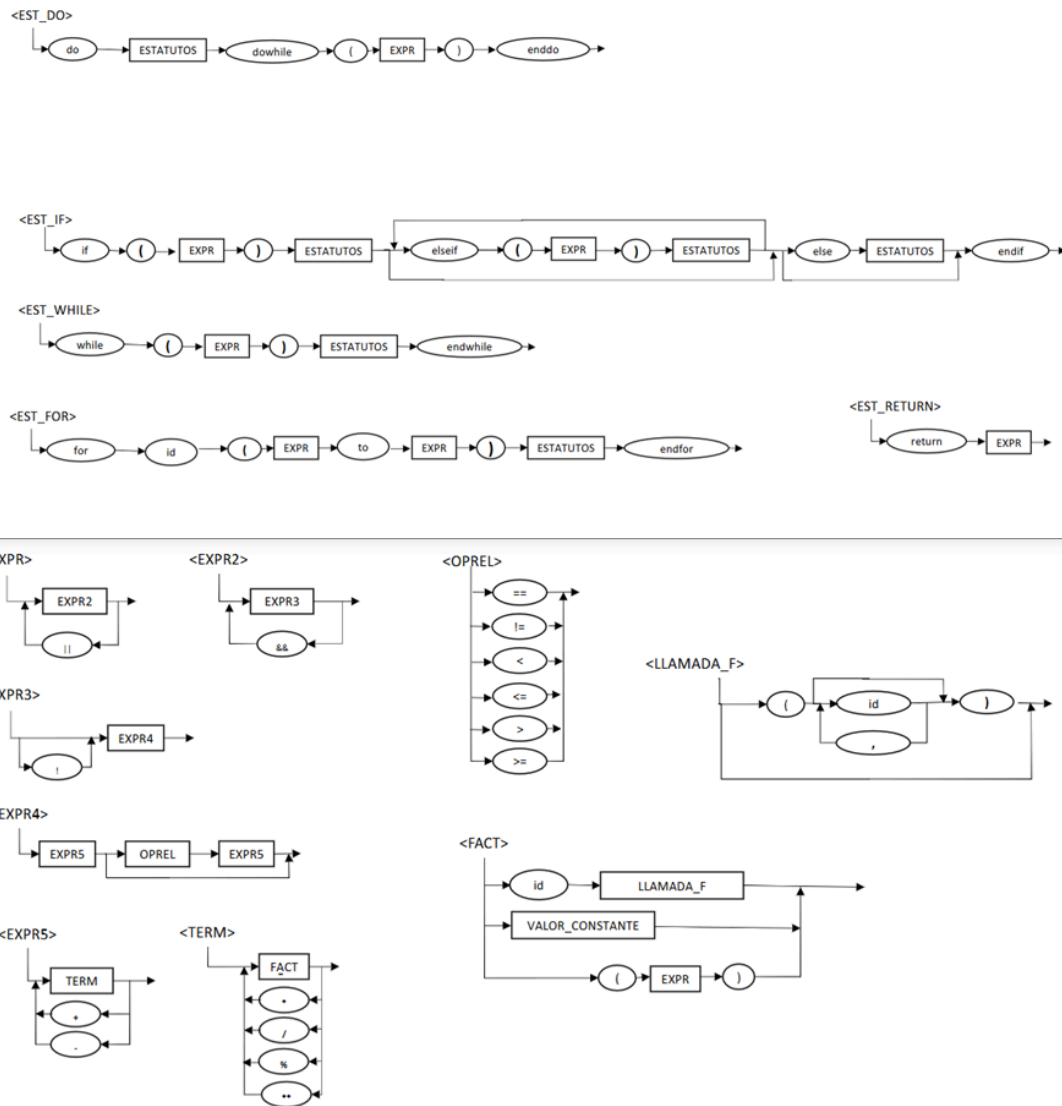
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
M	L	d	E	e	+	=	<	>	!	&	Comilla	'					
q0	1	2	3	506	134	2	1	19	20	9	10	11	12	13	14	15	17
q1	1	2	2	2	100	2	1	100	100	100	100	100	100	100	100	100	100
q2	2	2	2	2	101	2	2	101	101	101	101	101	101	101	101	101	101
q3	102	102	3	102	4	102	102	102	102	102	102	102	102	102	102	102	102
q4	500	500	5	500	500	500	500	500	500	500	500	500	500	500	500	500	500
q5	103	103	5	103	103	5	6	103	103	103	103	103	103	103	103	103	103
q6	501	501	8	501	501	501	501	7	7	501	501	501	501	501	501	501	501
q7	502	502	8	502	502	502	502	502	502	502	502	502	502	502	502	502	502
q8	104	104	8	104	104	104	104	104	104	104	104	104	104	104	104	104	104
q9	109	109	109	109	109	109	109	109	109	110	109	109	109	109	109	109	109
q10	111	111	111	111	111	111	111	111	111	112	111	111	111	111	111	111	111
q11	113	113	113	113	113	113	113	113	113	114	113	113	113	113	113	113	113
q12	116	116	116	116	116	116	116	116	116	115	116	116	116	116	116	116	116
q13	503	503	503	503	503	503	503	503	503	503	503	503	503	503	117	503	503
q14	504	504	504	504	504	504	504	504	504	504	504	504	504	504	118	504	504
q15	16	16	16	16	16	16	16	16	16	16	16	16	16	16	16	505	16
q16	507	507	507	507	507	507	507	507	507	507	507	507	507	507	507	507	507
q17	17	17	17	17	17	17	17	17	17	17	17	17	17	17	17	17	17
q18	126	126	126	126	126	126	126	126	126	126	126	126	126	126	126	126	17
q19	105	105	105	105	105	105	105	105	105	105	105	105	105	105	105	105	105
q20	106	106	106	106	106	106	106	106	106	106	106	106	106	106	106	106	106
q21	108	108	108	108	108	108	108	108	108	108	108	108	108	108	108	108	108
q22	22	22	22	22	22	22	22	22	22	22	22	22	22	22	22	22	22
q23	23	23	23	23	23	23	23	23	23	23	23	23	23	23	23	23	23
q24	23	23	23	23	23	23	23	23	23	23	23	23	23	23	23	23	23
q25	107	107	107	107	107	107	107	107	107	107	107	107	107	107	107	107	107

	17	18	19	20	21	22	23	24	25	26	27	28	29	30
%	/	*	#	(	)	:	.	dif #	/t	/n	/b			
7	128	21	25	127	119	120	121	122	123	124	506	0	0	0
7	100	100	100	100	100	100	100	100	100	100	100	100	100	100
L	101	101	101	101	101	101	101	101	101	101	101	101	101	101
2	102	102	102	102	102	102	102	102	102	102	102	102	102	102
0	500	500	500	500	500	500	500	500	500	500	500	500	500	500
3	103	103	103	103	103	103	103	103	103	103	103	103	103	103
L	501	501	501	501	501	501	501	501	501	501	501	501	501	501
2	502	502	502	502	502	502	502	502	502	502	502	502	502	502
2	104	104	104	104	104	104	104	104	104	104	104	104	104	104
3	109	109	109	109	109	109	109	109	109	109	109	109	109	109
L	111	111	111	111	111	111	111	111	111	111	111	111	111	111
3	112	113	113	113	113	113	113	113	113	113	113	113	113	113
3	116	116	116	116	116	116	116	116	116	116	116	116	116	116
3	503	503	503	503	503	503	503	503	503	503	503	503	503	503
3	504	504	504	504	504	504	504	504	504	504	504	504	504	504
3	16	16	16	16	16	16	16	16	16	16	16	16	16	16
7	507	507	507	507	507	507	507	507	507	507	507	507	507	507
3	17	17	17	17	17	17	17	17	17	17	17	17	17	17
7	126	126	126	126	126	126	126	126	126	126	126	126	126	126
5	105	105	105	105	105	105	105	105	105	105	105	105	105	105
3	106	106	106	106	106	106	106	106	106	106	106	106	106	106
3	108	22	23	108	108	108	108	108	108	108	108	108	108	108
2	22	22	22	22	22	22	22	22	22	22	22	22	131	22
3	23	23	24	23	23	23	23	23	23	23	23	23	23	23
3	23	132	24	23	23	23	23	23	23	23	23	23	23	23
7	107	107	133	107	107	107	107	107	107	107	107	107	107	107

ANALIZADOR SINTÁCTICO







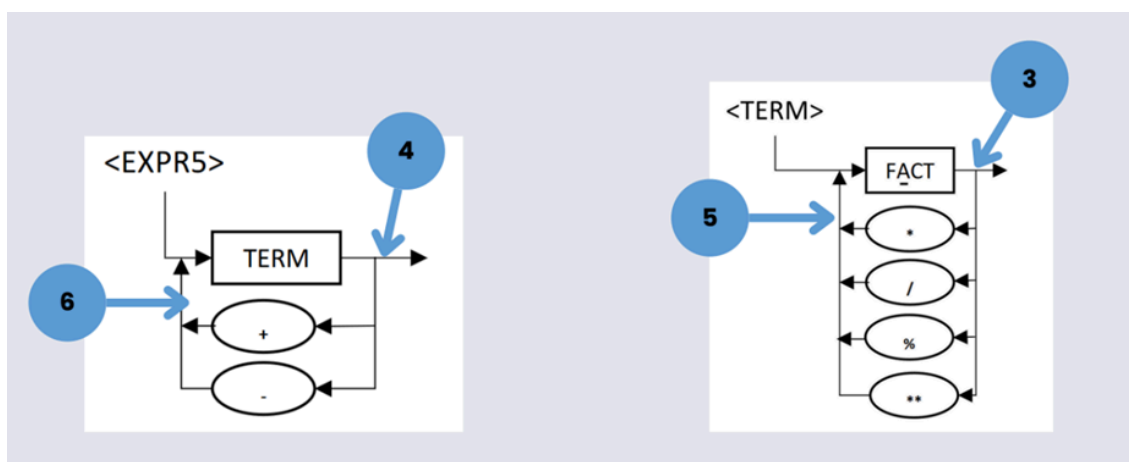
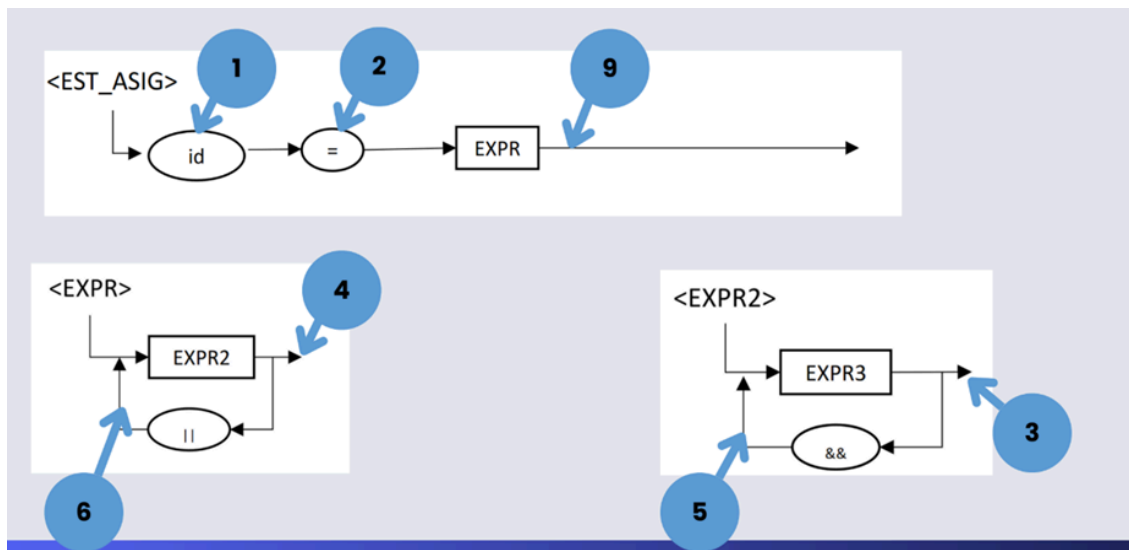
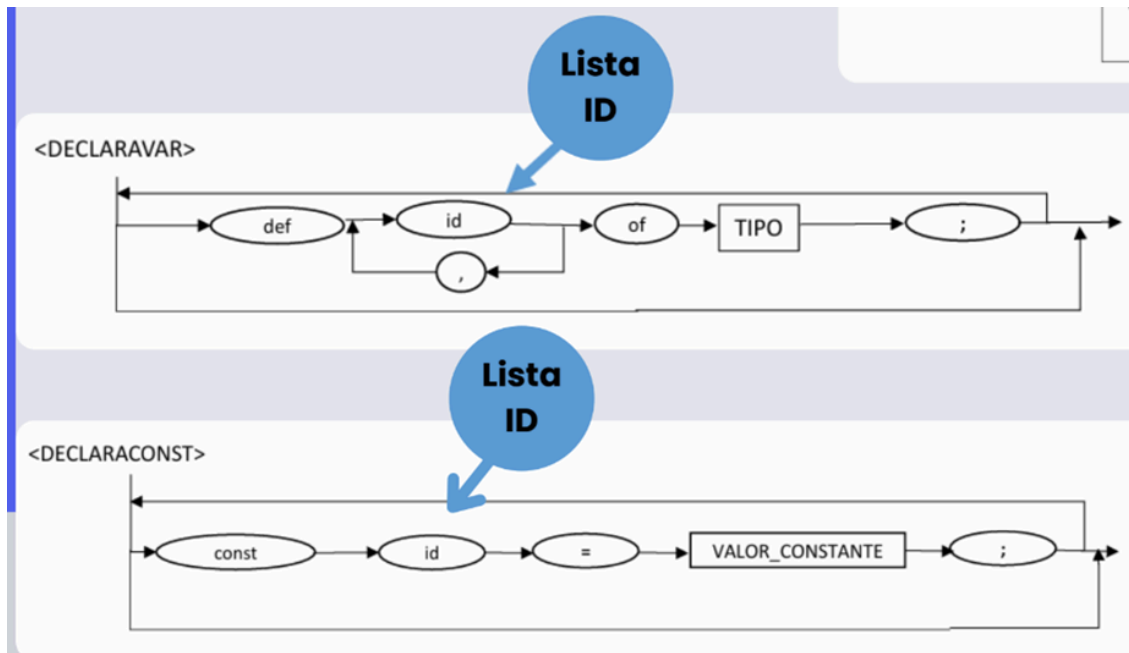
Matriz Predictiva

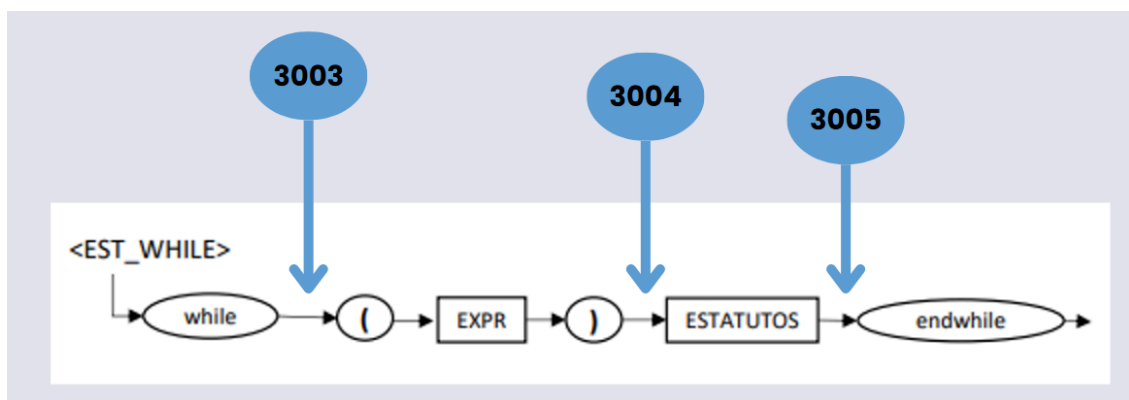
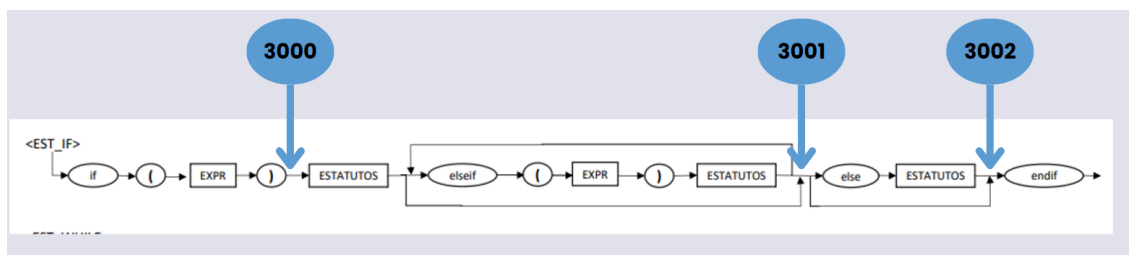
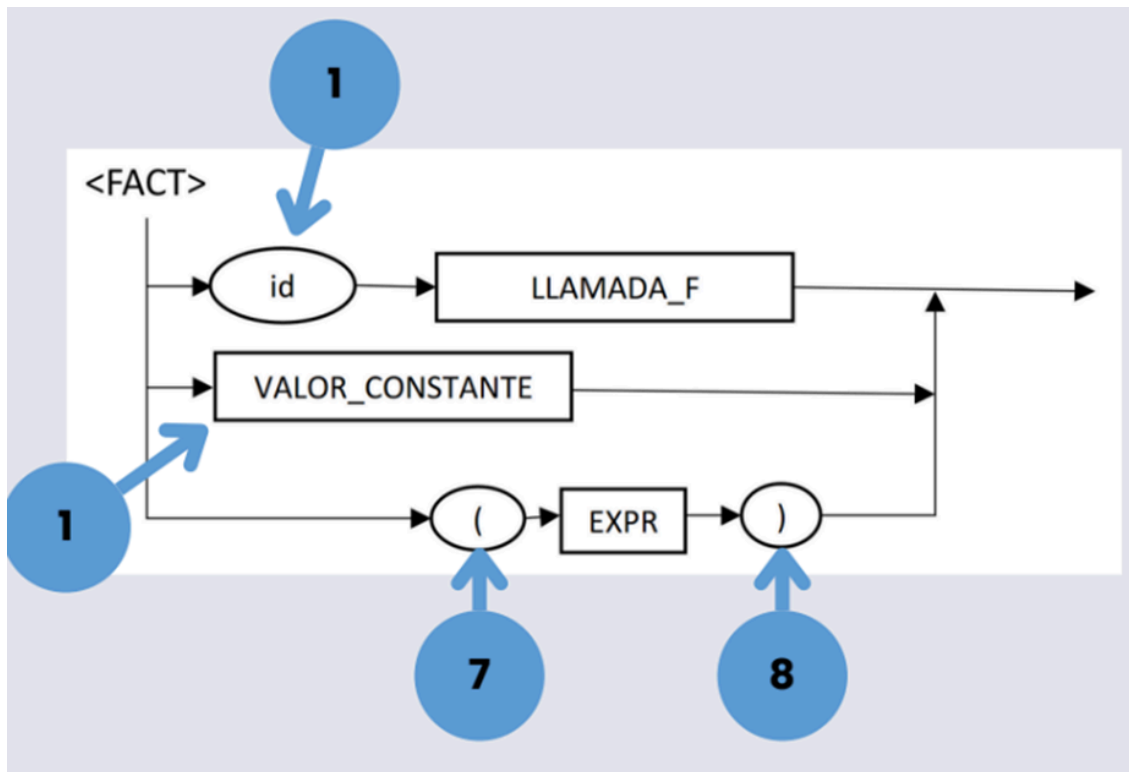
Hecha a base con las producciones semánticas obtenidas de los diagramas de sintaxis con los cuales se les hizo unas modificaciones en el proceso, donde su función es vincular cada token generado por el analizador léxico con el cual corresponde a la producción que se usa y que mas adelante sirve para la matriz predictiva.

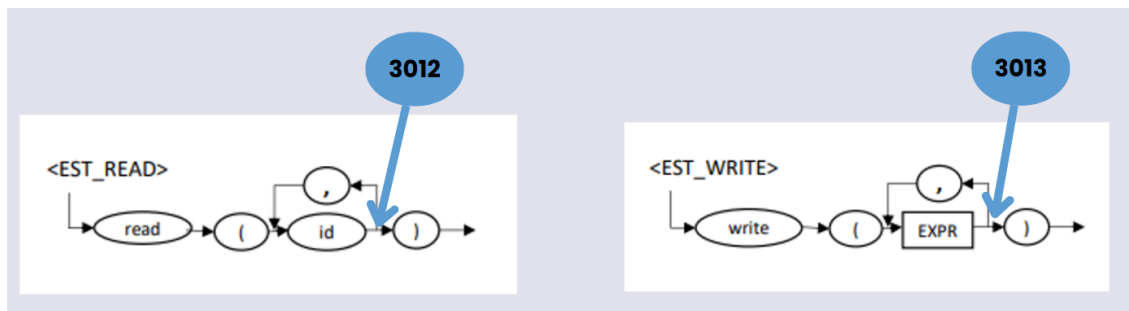
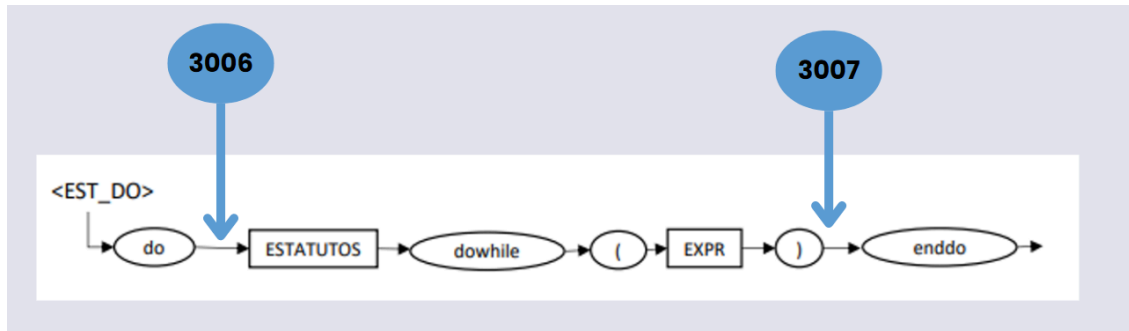
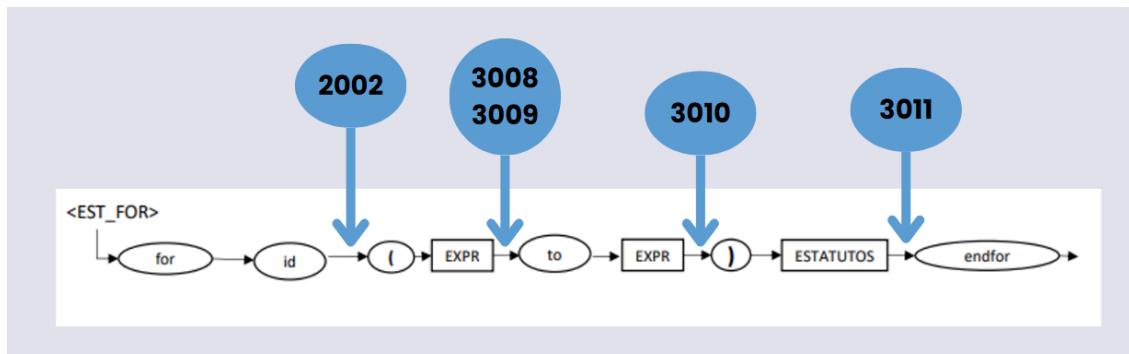
[illegible]

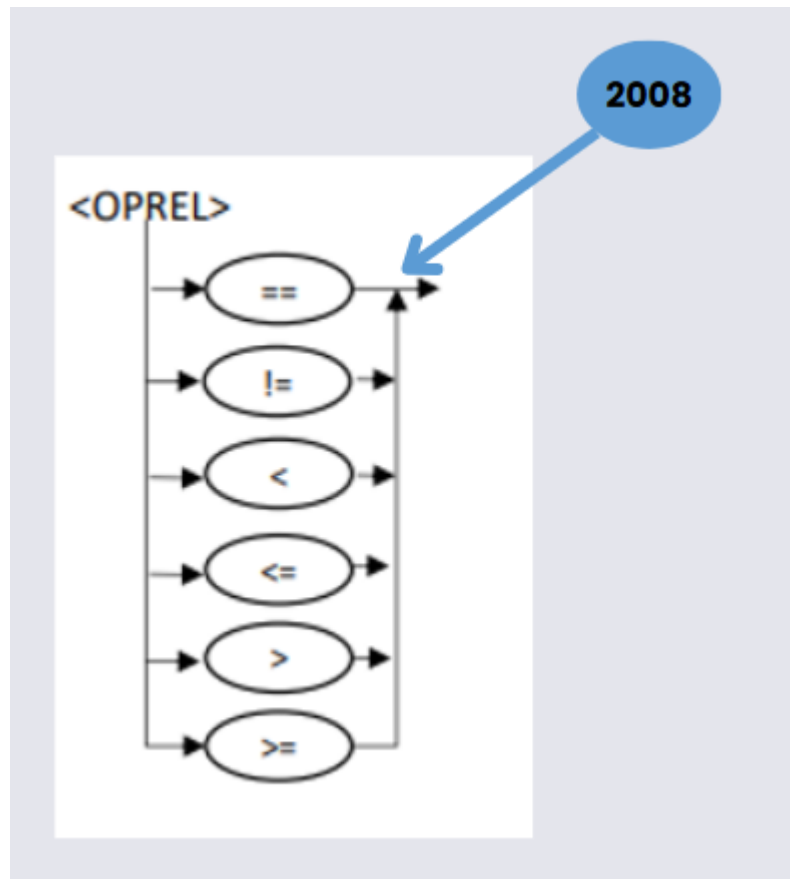
El análisis semántico es la tercera fase del compilador, posterior al análisis léxico y sintáctico. Su objetivo principal es verificar el significado y la coherencia del código fuente, asegurando que las instrucciones tengan sentido lógico y respeten las reglas del lenguaje, especialmente en cuanto a los tipos de datos y el uso de identificadores.

- Verificar que las variables sean declaradas antes de ser usadas.









Componentes Principales

1. Clase SemanticAnalyzer

Esta clase centraliza toda la lógica semántica. Es responsable de gestionar la tabla de símbolos, las pilas semánticas necesarias para evaluar expresiones y ejecutar las acciones semánticas disparadas por el parser.

headers/semantic_analyzer.h (Estructura de la Clase):

```
// Estructura para cada entrada en la Tabla de Simbolos
struct Simbolo {
    std::string tipo;
    int direccion;

};

#define FONDO_FALSO 999
```

```

class SemanticAnalyzer {
private:
    std::unordered_map<std::string, Simbolo> tablaSimbolos;
    std::stack<std::string> pilaLexemas;
    int cont_direcc; // Contador para asignar "direcciones" de memoria
    //pilas para expresiones
    std::stack<std::string> pilaTipos;
    std::stack<int> pilaOperadores;
    std::vector<std::string>* pListaErrores;

public:
    SemanticAnalyzer();

    // ACCIONES SEMANTICAS

    // Funcion para conectar la lista de errores
    void setReferenciaErrores(std::vector<std::string>* refLista);
    void reset();

    // Accion 1 (como la AccionId1): Revisa y apila el lexema del ID
    void accionDeclararId(const std::string& lexema);

    // Accion 2 (como la AccionId2): Desapila y registra en la tabla con un tipo
    void accionAsignarTipo(const std::string& tipo);

    // (Accion 1 → 2003) y mi sugerencia (2002)
    void accionPushTipoConstante(int tokenEstado);
    void accionVerificarIdYPushTipo(const std::string& lexema);

    // (Accion 2, 5, 6 → 2004, 2007, 2008)
    void accionPushOperador(int opTokenEstado);

    // (Accion 3, 4 → 2005, 2006)
    void accionResolverOpsPrecedencia(int precedencia);

```

```

// (Accion 7, 8 → 2009, 2010)
void accionPushFondoFalso();
void accionPopFondoFalso();

// (Accion 9 → 2011)
void accionResolverAsignacion();

//FUNCIONES AUXILIARES

// Convierte un token de tipo (140, 141...) a un string ("int", "float"... )
std::string getTipoFromToken(int tokenEstado);
// Convierte token cte a string
std::string getTipoConstanteFromToken(int tokenEstado);
// Para depuración
void imprimirTablaSimbolos();

private:
// Logica interna para usar el Cubo Semantico
void resolverUnaOperacion();
// Convierte token op a string
std::string opTokenToString(int opTokenEstado);

};

#endif // SEMANTIC_ANALYZER_H

```

Define la interfaz y los miembros privados de **SemanticAnalyzer**. Incluye la **tablaSimbolos**, las pilas (**pilaLexemas**, **pilaTipos**, **pilaOperadores**), el contador de direcciones (**cont_direcc**), el puntero para reportar errores (**pListaErrores**) y los prototipos de todas las acciones semánticas y funciones auxiliares.

2. Tabla de Símbolos (**tablaSimbolos** y **struct Simbolo**)

Es una estructura de datos fundamental (en este caso, un **unordered_map**) que almacena información sobre cada identificador (variable) encontrado en el código. La información clave guardada es el **tipo de dato** y una **dirección de memoria virtual** asignada.

headers/semantic_analyzer.h (Definición de Simbolo):

```
// Estructura para cada entrada en la Tabla de Simbolos
struct Simbolo {
    std::string tipo; // Ej: "int", "float", "char"
    int direccion; // Direccion de memoria virtual asignada
    // ...se pueden añadir más campos
};
```

Define la estructura **Simbolo** que representa la información asociada a cada identificador en la tabla. Cada **Simbolo** contiene al menos el **tipo** de dato y una **direccion** única.

headers/semantic_analyzer.h (Declaración de **tablaSimbolos**)

```
class SemanticAnalyzer {
private:
    std::unordered_map<std::string, Simbolo> tablaSimbolos;
```

Declara la **tablaSimbolos** como un mapa donde la clave es el nombre del identificador (el **string** del lexema) y el valor es la estructura **Simbolo** con su información.

sources/semantic_analyzer.cpp (Uso en **accionAsignarTipo**):

```
/*
 * Accion 2 (Pop y registrar)
 * Llama cuando el parser ve el tipo de datos (of int, of float...)
 */
void SemanticAnalyzer::accionAsignarTipo(const std::string& tipo) {
    qDebug() << "Acción (2001): Registrando variables con tipo '" << QString::fromStdString(tipo) << "'";
    // 1. Sacar todos los IDs de la pila temporal
    while (!pilaLexemas.empty()) {
        std::string lexema_act = pilaLexemas.top();
        pilaLexemas.pop();
        int nuevaDirecc = cont_direcc++;
        Simbolo nuevoSimbolo = { tipo, nuevaDirecc };
    }
```

```

        tablaSimbolos.emplace(lexema_act, nuevoSimbolo);
        qDebug() << " → Registrado '" << QString::fromStdString(lexema_act)
<< "' (Tipo: " << QString::fromStdString(tipo) << "));
    }
}

```

Muestra cómo se inserta un nuevo símbolo en la tabla cuando se procesa una declaración de variable. Se le asigna el tipo correspondiente y una nueva dirección.

Así como las Acciones correspondientes; Estas funciones son invocadas por el analizador sintáctico (**Parser**) cuando encuentra los marcadores numéricos correspondientes (2000-2012) durante el análisis de la gramática. Cada función realiza una tarea específica del análisis semántico.

```

/*
 * Accion 1 (Push ID)
 * Llama cuando el parser ve un ID en una declaracion
 */
void SemanticAnalyzer::accionDeclararId(const std::string& lexema) {
    // 1. Revisar si ya existe (Regla de no-redeclaracion)
    if (tablaSimbolos.count(lexema)) {
        std::string errorMsg = "ERROR SEMÁNTICO: El identificador '" + lexema + "' ya ha sido declarado.";
        qDebug() << QString::fromStdString(errorMsg); // Dejamos el debug
        if (pListaErrores) {
            pListaErrores->push_back(errorMsg); // se añade a la lista
        }
    } else {
        qDebug() << "Acción (2000): Apilando ID para declarar '" << QString::fromStdString(lexema) << "'";
        pilaLexemas.push(lexema);
    }
}
}

```

Funcion:

- Se ejecuta después de que el parser reconoce un identificador (**id**) dentro de una sentencia de declaración (**def**).
- Recibe el nombre del identificador (**lexema**).
- **Verifica duplicados:** Consulta la **tablaSimbolos** principal para verificar si ese **lexema** ya fue declarado (**tablaSimbolos.count(lexema)**).
- **Reporta el error:** Si el identificador ya existe, construye un mensaje de error, lo muestra en la consola (**qDebug**) y lo añade a la lista de errores (**pListaErrores**) que se mostrará en la interfaz gráfica.
- **Apilado temporal:** Si el identificador es nuevo, lo inserta (**push**) en la **pilaLexemas**. Esta pila almacena temporalmente los identificadores encontrados en una misma línea de declaración (ej. **def var1, var2 of int;**) antes de conocer su tipo.
- Prepara la recepción de **2001 (accionAsignarTipo)**, que vacía **pilaLexemas** y registra los identificadores en la tabla de símbolos según su tipo.

```
// (2002)
void SemanticAnalyzer::accionVerificarIdYPushTipo(const std::string& lexema) {
    //QDebug() << "Acción (2002): Verificando ID '" << QString::fromStdString(lexema) << "'";
    auto it = tablaSimbolos.find(lexema);
    if (it == tablaSimbolos.end()) {
        // 1. Mandar el error original
        std::string errorMsg = "ERROR SEMÁNTICO: Variable no declarada '"
+ lexema + "'";
        qDebug().noquote() << errorMsg;
        if (pListaErrores) {
            pListaErrores->push_back(errorMsg);
        }
        // 2. se parchea con un real
        pilaTipos.push(T_FLOAT);
        qDebug().nospace().noquote() << "Push Tipo: [" << T_FLOAT << "];"
    } else {
        qDebug().noquote() << "Acción (2002/12): Verificando ID '" << QString::fromStdString(lexema) << "'";
    }
}
```

```

        pilaTipos.push(it→second.tipo);
        qDebug().nospace().noquote() << "Push Tipo: [" << QString::fromStdS
tring(it→second.tipo) << "];";
    }
    debugImprimirPilaTipos(pilaTipos);
}

```

Función:

- Se ejecuta cuando el parser encuentra un identificador que se está *usando* (no declarando), ya sea en el lado derecho de una expresión (Acción 2002) o en el lado izquierdo de una asignación (Acción 2012).
- Recibe el nombre del identificador (**lexema**).
- **Verifica existencia:** Busca el **lexema** en la **tablaSimbolos** mediante **tablaSimbolos.find(lexema)**.
- **Reporte de error:** Si no lo encuentra (**it == tablaSimbolos.end()**), la variable no fue declarada. Reporta el error en la consola y en la lista **pListaErrores**. Para continuar el análisis, apila y parchea como float (real) en la **pilaTipos**. (Variable no definida, aquí detecta el error)
- **Apilado de tipo:** Si el identificador existe, recupera su tipo (**it→second.tipo**) de la tabla y lo mete en la **pilaTipos**.

```

// (Accion 1 → 2003)
void SemanticAnalyzer::accionPushTipoConstante(int tokenEstado) {
    std::string tipo = getTipoConstanteFromToken(tokenEstado);
    qDebug() << "Acción (2003): Push tipo constante '" << QString::fromStd
String(tipo) << "'";
    pilaTipos.push(tipo);
}

```

Función:

- Se activa cuando el parser reconoce una constante literal (un número entero, real, carácter o cadena).

- Recibe el número (**tokenEstado**) que el analizador léxico asignó a ese tipo de constante (ej. 102 para entero, 126 para string).
- **Determina tipo:** Llama a la función auxiliar **getTipoConstanteFromToken** para traducir el **tokenEstado** léxico al string del tipo correspondiente (**T_INT**, **T_FLOAT**, etc.).
- **Apilado de tipo:** Mete el string del tipo obtenido en la **pilaTipos**.

```
// (Accion 2, 5, 6 → 2004, 2007, 2008)
void SemanticAnalyzer::accionPushOperador(int opTokenEstado) {
    qDebug() << "Acción (200x): Push operador '" << opTokenEstado << "'";
    pilaOperadores.push(opTokenEstado);
}
```

Función:

- Se ejecuta cuando el parser reconoce un operador aritmético, lógico, relacional o de asignación.
- Recibe el número (**opTokenEstado**) que el léxico asignó a ese operador (ej. 109 para =, 107 para , 105 para +).
- **Apilado de operador:** Mete directamente el **opTokenEstado** numérico en la **pilaOperadores**.

```
// (Accion 3, 4 → 2005, 2006)
// Usamos un truco: 2005 (Accion 3) es para precedencia alta (*, /)
// 2006 (Accion 4) es para precedencia baja (+, -)
void SemanticAnalyzer::accionResolverOpsPrecedencia(int precedencia) {
    qDebug() << "Acción (2005/2006): Resolviendo precedencia..." ;
    while (!pilaOperadores.empty()) {
        int op = pilaOperadores.top();

        // precedencia 1 = alta (*, /)
        // precedencia 2 = baja (+, -)
        if (precedencia == 1 && (op == OP_MULT || op == OP_DIV || op == OP_MOD || op == OP_POT)) {
```

```

        resolverUnaOperacion();
    } else if (precedencia == 2 && (op == OP_SUMA || op == OP_RESTA ||
op == OP_OR)) {
        resolverUnaOperacion();
    } else {
        break; // No hay mas operaciones de esta precedencia
    }
}
}
}

```

Función:

- Implementa las reglas de precedencia de operadores. Se invoca en puntos estratégicos de la gramática para garantizar que las multiplicaciones se resuelvan antes que las sumas.
- Recibe un nivel de **precedencia** (1 para alta: *, /, etc.; 2 para baja: +, -, etc.).
- **Bucle de resolución:** Mientras la **pilaOperadores** no esté vacía, examina el operador en la cima con **top()**.
- **Verifica precedencia:** Comprueba si el operador en la cima corresponde al nivel de **precedencia** solicitado.
- **Resuelve:** Si coincide la precedencia, llama a **resolverUnaOperacion()**, que extrae dos tipos y un operador, consulta el cubo semántico e inserta el tipo resultante.
- **Termina:** Si el operador en la cima tiene diferente precedencia o la pila se vacía, el bucle **while** termina.

```

// (Accion 7 → 2009)
void SemanticAnalyzer::accionPushFondoFalso() {
    qDebug() << "Acción (2009): Push Fondo Falso";
    pilaOperadores.push(FONDO_FALSO);
}

```

Función:

- Se ejecuta justo antes de que el parser procese un paréntesis de apertura (.).

- **Apila marcador:** Insertar el valor especial **FONDO_FALSO** (definido como **999**) en la **pilaOperadores**.

```
// (Accion 8 → 2010)
void SemanticAnalyzer::accionPopFondoFalso() {
    qDebug() << "Acción (2010): Pop Fondo Falso";
    // Resolver todas las operaciones pendientes dentro del parentesis
    while (!pilaOperadores.empty() && pilaOperadores.top() != FONDO_FALSO) {
        resolverUnaOperacion();
    }

    if (pilaOperadores.empty()) {
        qDebug() << "ERROR SINTÁCTICO: Falta paréntesis que abre '('.";
    } else {
        pilaOperadores.pop(); // Sacar el FONDO_FALSO
    }
}
```

Función:

- Se ejecuta inmediatamente después de que el parser procesa un paréntesis de cierre).
- **Resuelve operaciones internas:** Entra en un bucle **while** que evalúa todas las operaciones pendientes en la **pilaOperadores** llamando a **resolverUnaOperacion()**, hasta encontrar el **FONDO_FALSO** o vaciar la pila. Esto garantiza que toda la expresión dentro del paréntesis se calcule correctamente.
- **Verifica y extrae el marcador:** Comprueba si el **FONDO_FALSO** está presente. Si no está, indica un error de paréntesis desbalanceado. Si está presente, lo extrae (**pop**) de la **pilaOperadores**, eliminando el delimitador.

```
// (Accion 9 → 2011)
void SemanticAnalyzer::accionResolverAsignacion() {
```

```

qDebug() << "Acción (2011): Resolviendo asignación...";
resolverUnaOperacion(); // La asignacion es solo otra operacion
}

```

Función:

- Se ejecuta al final de una sentencia de asignación, después de evaluar toda la expresión del lado derecho. Su tipo resultante debe estar en la cima de **pilaTipos**.
- **Delega resolución:** Llama a **resolverUnaOperacion()**. En este punto, **pilaTipos** contiene [**tipo_izquierdo**, **tipo_derecho**] y **pilaOperadores** tiene el operador = (**OP_ASIG**) en el tope.
- **Verificación final:** **resolverUnaOperacion** extrae los dos tipos y el operador =, consulta el **CuboSemantico** para verificar si la asignación es válida (por ejemplo, **int = int** es válido, **string = float** no lo es), reporta errores si es necesario y apila el tipo resultante (o **T_ERROR**) de vuelta en **pilaTipos**.

3. Pilas Semánticas (pilaTipos, pilaOperadores)

Se utilizan dos pilas principales durante el análisis de expresiones para implementar el algoritmo de evaluación y chequeo de tipos:

- **pilaTipos:** Almacena los **tipos de dato** de los operandos a medida que se encuentran (**int**, **float**, **error_tipo**, etc.).
- **pilaOperadores:** Almacena los **operadores** (+, -, =, (, etc.) encontrados, gestionando su precedencia.

headers/semantic_analyzer.h (Declaración de las Pilas):

```

class SemanticAnalyzer {
private:
    std::unordered_map<std::string, Simbolo> tablaSimbolos;
    std::stack<std::string> pilaLexemas;
    int cont_direcc; // Contador para asignar "direcciones" de memoria
    //pilas para expresiones
    std::stack<std::string> pilaTipos;

```

```
std::stack<int> pilaOperadores;  
std::vector<std::string>* pListaErrores;
```

Declara las dos pilas como miembros privados de la clase **SemanticAnalyzer**.

sources/semantic_analyzer.cpp (Uso en **resolverUnaOperacion**):

```
void SemanticAnalyzer::resolverUnaOperacion() {  
    // 1. Sacar operador  
    int op = pilaOperadores.top();  
    pilaOperadores.pop();  
  
    // 2. Sacar tipo 2 (operando derecho)  
    std::string tipo2 = pilaTipos.top();  
    pilaTipos.pop();  
  
    // 3. Sacar tipo 1 (operando izquierdo)  
    std::string tipo1 = pilaTipos.top();  
    pilaTipos.pop();  
  
    // 4. Verificar en el Cubo Semantico  
    if (tipoResultado == T_ERROR) {  
        std::string errorMsg;  
  
        // Checamos si el operador es una asignación  
        if (op == OP_ASIG) { // token 109 (=)  
            errorMsg = "ERROR SEMÁNTICO: Tipos incompatibles en asignación (" + tipo1 + " " + opStr + " " + tipo2 + ")";  
        } else {  
            errorMsg = "ERROR SEMÁNTICO: Operación entre tipos incompatible (" + tipo1 + " " + opStr + " " + tipo2 + ")";  
        }  
  
        qDebug().noquote() << "      " << QString::fromStdString(errorMsg); //  
        Indentado  
        if (pListaErrores) {
```

```

        pListaErrores→push_back(errorMsg);
    }

    // Se parchea ponemos un real en lugar del error
    tipoResultado = T_FLOAT;
    qDebug().nospace().noquote() << " → PATCH: Result Type: [" << tipo
Resultado << "];";

    } else{
        qDebug().nospace().noquote() << "      = " << tipoResultado;
    }

    // 5. Meter el resultado de vuelta a la pila de tipos
    pilaTipos.push(tipoResultado);

    debugImprimirPilaTipos(pilaTipos);
    debugImprimirPilaOperadores(pilaOperadores);
}

```

Muestra cómo se usan las pilas para evaluar una operación: se sacan dos tipos y un operador, se verifica el tipo resultante, y este se vuelve a meter en la **pilaTipos**.

4. Cubo Semántico (CuboSemantico)

Es una clase estática que encapsula las **reglas de compatibilidad de tipos** para todas las operaciones binarias del lenguaje. Define qué tipo de dato resulta al aplicar un operador (+, *, =, <, etc.) a dos operandos de tipos específicos. Si una combinación no es válida, devuelve un tipo especial **T_ERROR**.

headers/cubo_semantico.h (Estructura de la Clase):

```

#include <string>
#include <unordered_map>
#include <QDebug>

// Definimos los operadores usando los numeros de token lexico

```

```

#define OP_SUMA 105
#define OP_RESTA 106
#define OP_MULT 107
#define OP_DIV 108
#define OP_POT 133
#define OP_MOD 128
#define OP_AND 117
#define OP_OR 118
#define OP_ASIG 109

// Definimos tipos para claridad
#define T_INT "int"
#define T_FLOAT "float"
#define T_CHAR "char"
#define T_STRING "string"
#define T_BOOL "bool"
#define T_ERROR "error_tipo"

// Relacionales
#define OP_IGUAL 110
#define OP_DIFERENTE 115
#define OP_MENOR 111
#define OP_MENOR_IGUAL 112
#define OP_MAYOR 113
#define OP_MAYOR_IGUAL 114

class CuboSemantico {
private:
    // Un mapa estatico para guardar las reglas: Es [Operador][Tipo1][Tipo
2] → TipoResultado
    static std::unordered_map<int,
                                std::unordered_map<std::string,
                                std::unordered_map<std::string, std::string
>>> cubo;

    static bool inicializado;
    static void inicializarCubo();

```

```

public:
    // La función principal que usa el analizador
    static std::string verificar(int op, const std::string& tipo1, const std::string& tipo2);
};

#endif // CUBO_SEMANTICO_H

```

Define la interfaz de **CuboSemantico**. Contiene un mapa estático (**cubo**) para almacenar las reglas y la función principal **verificar** que consulta el cubo.

sources/cubo_semantico.cpp (Ejemplo de Regla):

```

#include "cubo_semantico.h"

// Inicializacion de variables estaticas
std::unordered_map<int,
    std::unordered_map<std::string,
        std::unordered_map<std::string, std::string>>> CuboSemantico::cubo;
bool CuboSemantico::inicializado = false;

void CuboSemantico::inicializarCubo() {
    // --- Logica para SUMA (+)
    cubo[OP_SUMA][T_INT][T_INT] = T_INT;
    cubo[OP_SUMA][T_INT][T_FLOAT] = T_FLOAT;
    cubo[OP_SUMA][T_FLOAT][T_INT] = T_FLOAT;
    cubo[OP_SUMA][T_FLOAT][T_FLOAT] = T_FLOAT;
    cubo[OP_SUMA][T_STRING][T_STRING] = T_STRING; // Concatenar
    cubo[OP_SUMA][T_STRING][T_INT] = T_STRING;
    cubo[OP_SUMA][T_INT][T_STRING] = T_STRING;
    //todas las demas sumas son T_ERROR

    // --- Logica para RESTA (-)
    cubo[OP_RESTA][T_INT][T_INT] = T_INT;
    cubo[OP_RESTA][T_INT][T_FLOAT] = T_FLOAT;
    cubo[OP_RESTA][T_FLOAT][T_INT] = T_FLOAT;

```

```

cubo[OP_RESTA][T_FLOAT][T_FLOAT] = T_FLOAT;

// --- Logica para MULTIPLICACION (*)
cubo[OP_MULT][T_INT][T_INT] = T_INT;
cubo[OP_MULT][T_INT][T_FLOAT] = T_FLOAT;
cubo[OP_MULT][T_FLOAT][T_INT] = T_FLOAT;
cubo[OP_MULT][T_FLOAT][T_FLOAT] = T_FLOAT;

// --- Logica para DIVISION (/)
cubo[OP_DIV][T_INT][T_INT] = T_FLOAT; // int/int → float (para evitar
perdida)
cubo[OP_DIV][T_INT][T_FLOAT] = T_FLOAT;
cubo[OP_DIV][T_FLOAT][T_INT] = T_FLOAT;
cubo[OP_DIV][T_FLOAT][T_FLOAT] = T_FLOAT;

// --- Logica para ASIGNACION (=)
cubo[OP_ASIG][T_INT][T_INT] = T_INT;
cubo[OP_ASIG][T_FLOAT][T_FLOAT] = T_FLOAT;
cubo[OP_ASIG][T_FLOAT][T_INT] = T_FLOAT; // Permite int → float
cubo[OP_ASIG][T_CHAR][T_CHAR] = T_CHAR;
cubo[OP_ASIG][T_STRING][T_STRING] = T_STRING;
cubo[OP_ASIG][T_BOOL][T_BOOL] = T_BOOL;
// Asignar float a int es T_ERROR (requeriria casteo explicito)

// --- Logica para AND (&&) y OR (||)
cubo[OP_AND][T_BOOL][T_BOOL] = T_BOOL;
cubo[OP_OR][T_BOOL][T_BOOL] = T_BOOL;

// --- Logica para mod
cubo[OP_MOD][T_INT][T_INT] = T_INT;

// --- Logica para pot
cubo[OP_POT][T_INT][T_INT] = T_FLOAT;
cubo[OP_POT][T_INT][T_FLOAT] = T_FLOAT;
cubo[OP_POT][T_FLOAT][T_INT] = T_FLOAT;
cubo[OP_POT][T_FLOAT][T_FLOAT] = T_FLOAT;

```

```

// --- Logica para relacionales

// Operador == (110)
cubo[OP_IGUAL][T_INT][T_INT] = T_BOOL;
cubo[OP_IGUAL][T_FLOAT][T_FLOAT] = T_BOOL;
cubo[OP_IGUAL][T_INT][T_FLOAT] = T_BOOL;
cubo[OP_IGUAL][T_FLOAT][T_INT] = T_BOOL;
cubo[OP_IGUAL][T_CHAR][T_CHAR] = T_BOOL;
cubo[OP_IGUAL][T_STRING][T_STRING] = T_BOOL;
cubo[OP_IGUAL][T_BOOL][T_BOOL] = T_BOOL;

// Operador != (115)
cubo[OP_DIFERENTE][T_INT][T_INT] = T_BOOL;
cubo[OP_DIFERENTE][T_FLOAT][T_FLOAT] = T_BOOL;
cubo[OP_DIFERENTE][T_INT][T_FLOAT] = T_BOOL;
cubo[OP_DIFERENTE][T_FLOAT][T_INT] = T_BOOL;
cubo[OP_DIFERENTE][T_CHAR][T_CHAR] = T_BOOL;
cubo[OP_DIFERENTE][T_STRING][T_STRING] = T_BOOL;
cubo[OP_DIFERENTE][T_BOOL][T_BOOL] = T_BOOL;

// Operador < (111)
cubo[OP_MENOR][T_INT][T_INT] = T_BOOL;
cubo[OP_MENOR][T_FLOAT][T_FLOAT] = T_BOOL;
cubo[OP_MENOR][T_INT][T_FLOAT] = T_BOOL;
cubo[OP_MENOR][T_FLOAT][T_INT] = T_BOOL;
cubo[OP_MENOR][T_CHAR][T_CHAR] = T_BOOL;
cubo[OP_MENOR][T_STRING][T_STRING] = T_BOOL;
cubo[OP_MENOR][T_BOOL][T_BOOL] = T_BOOL;

// Operador <= (112)
cubo[OP_MENOR_IGUAL][T_INT][T_INT] = T_BOOL;
cubo[OP_MENOR_IGUAL][T_FLOAT][T_FLOAT] = T_BOOL;
cubo[OP_MENOR_IGUAL][T_INT][T_FLOAT] = T_BOOL;
cubo[OP_MENOR_IGUAL][T_FLOAT][T_INT] = T_BOOL;
cubo[OP_MENOR_IGUAL][T_CHAR][T_CHAR] = T_BOOL;
cubo[OP_MENOR_IGUAL][T_STRING][T_STRING] = T_BOOL;
cubo[OP_MENOR_IGUAL][T_BOOL][T_BOOL] = T_BOOL;

```

```

// Operador > (113)
cubo[OP_MAYOR][T_INT][T_INT] = T_BOOL;
cubo[OP_MAYOR][T_FLOAT][T_FLOAT] = T_BOOL;
cubo[OP_MAYOR][T_INT][T_FLOAT] = T_BOOL;
cubo[OP_MAYOR][T_FLOAT][T_INT] = T_BOOL;
cubo[OP_MAYOR][T_CHAR][T_CHAR] = T_BOOL;
cubo[OP_MAYOR][T_STRING][T_STRING] = T_BOOL;
cubo[OP_MAYOR][T_BOOL][T_BOOL] = T_BOOL;

//Operador >= (114)
cubo[OP_MAYOR_IGUAL][T_INT][T_INT] = T_BOOL;
cubo[OP_MAYOR_IGUAL][T_FLOAT][T_FLOAT] = T_BOOL;
cubo[OP_MAYOR_IGUAL][T_INT][T_FLOAT] = T_BOOL;
cubo[OP_MAYOR_IGUAL][T_FLOAT][T_INT] = T_BOOL;
cubo[OP_MAYOR_IGUAL][T_CHAR][T_CHAR] = T_BOOL;
cubo[OP_MAYOR_IGUAL][T_STRING][T_STRING] = T_BOOL;
cubo[OP_MAYOR_IGUAL][T_BOOL][T_BOOL] = T_BOOL;

inicializado = true;
}

```

Muestra cómo se definen las reglas dentro de **inicializarCubo**. Por ejemplo, **int + int** da **int**, **int + float** da **float**, y asignar un **int** a un **float** es válido y el resultado es **float**. Combinaciones no definidas resultarán en **T_ERROR**.

5. Acciones Semánticas (Puntos de Inserción en Gramática)

Son marcadores numéricos (empezando desde 2000) insertados directamente en las **reglas de producción** de la gramática (en **parser_productions.cpp**). Cuando el parser encuentra uno de estos marcadores en la pila sintáctica, detiene momentáneamente el análisis sintáctico para ejecutar una función específica en **SemanticAnalyzer**. Esto permite realizar chequeos y manipulaciones semánticas en el momento preciso.

sources/parser_productions.cpp (Ejemplo con Acciones):

```

void Productions::inicializarProducciones() {
    producciones = {
        // ...
        // Declaración de variables:
        {12, {1002, 1000, 2000, 8, 1034, 9, 2001, 1037, 7}}, //<DECLARAVAR>
        //      id ^Acción    Tipo ^Acción
        //      PushID      AsignarTipo

        // Uso de ID en asignación (lado izquierdo):
        {47, {1000, 2012}}, //<PROD_ID>
        //      id ^Acción Verificar y Push Tipo L-Value

        // Uso de ID en expresión (lado derecho):
        {100, {1000, 2002, 44}}, //<FACT>
        //      id ^Acción Verificar y Push Tipo R-Value

        // Uso de constante en expresión:
        {101, {10, 2003}}, //<FACT>
        //      Cte ^Acción Push Tipo Constante
        // ...
    };
}

```

Aquí se muestra cómo se insertan los números de acción (ej: 2000, 2001, 2012, 2002, 2003) en las producciones gramaticales. La posición de la acción es muy importante porque si esta mal colocada no funciona; va *después* del token que necesita procesar y esto se hizo conforme al diagrama con las acciones.

Esta es la vista completa:

```

void Productions::inicializarProducciones() {
    producciones = {
        {1, {5, 2, 4}}, //program
        {2, {3, 2}}, //declaraciones
        {3, {-1}}, //declaraciones
        {4, {7}}, //desclaraciones 2
        {5, {6}}, //desclaraciones 2
        {6, {14}}, //desclaraciones 2
    };
}

```

```

{7, {1005, 15, 1026}}, //declara_class
{8, {1001, 1000, 1036, 1033, 1037, 5}}, //<DECLARA_LIBRARY>
{9, {-1}}, //<DECLARA_LIBRARY>
{10, {1003, 1000, 2000, 1039, 10, 1037, 6}}, //<DECLARACONST>
{11, {-1}}, //<DECLARACONST>
{12, {1002, 1000, 2000, 8, 1034, 9, 2001, 1037, 7}}, //<DECLARAVAR>
{13, {-1}}, //<DECLARAVAR>
{14, {1038, 1000, 2000, 8}}, //<ID2>
{15, {-1}}, //<ID2>
{16, {1006}}, //<TIPO_DE_DATO>
{17, {1007}}, //<TIPO_DE_DATO>
{18, {1008}}, //<TIPO_DE_DATO>
{19, {1009}}, //<TIPO_DE_DATO>
{20, {1010}}, //<TIPO_DE_DATO>
{21, {1011}}, //<TIPO_DE_DATO>
{22, {1012}}, //<VALOR_CONSTANTE>
{23, {1013}}, //<VALOR_CONSTANTE>
{24, {1014}}, //<VALOR_CONSTANTE>
{25, {1015}}, //<VALOR_CONSTANTE>
{26, {1016}}, //<VALOR_CONSTANTE>
{27, {1000, 13, 1039, 9, 12}}, //<PARAMETROS>
{28, {-1}}, //<PARAMETROS>
{29, {1038, 11}}, //<PARAMETROS2>
{30, {-1}}, //<PARAMETROS2>
{31, {1038, 1000, 2000, 13}}, //ID3
{32, {-1}}, //ID3
{33, {1004, 1000, 1039, 9, 1040, 11, 1041, 7, 15, 1027, 14}}, //<DECLARA_
FUNCTION>
{34, {-1}}, //<DECLARA_FUNCTION>
{35, {16, 1037, 15}}, //<ESTATUTOS>
{36, {-1}}, //<ESTATUTOS>
{37, {17}}, //<ESTATUTOS 2>
{38, {31}}, //<ESTATUTOS 2>
{39, {20}}, //<ESTATUTOS 2>
{40, {22}}, //<ESTATUTOS 2>
{41, {23}}, //<ESTATUTOS 2>
{42, {26, 15}}, //<ESTATUTOS>
{43, {29, 15}}, //<ESTATUTOS>

```

```

{44, {25, 15}}, //<ESTATUTOS>
{45, {30, 15}}, //<ESTATUTOS>
{46, {18, 19}}, //<EST_ASIG>
{47, {1000, 2012}}, //<PROD_ID>
{48, {1039, 2004, 32, 2011}}, //<PROD_ID2>
{49, {1047}}, //<PROD_ID2>
{50, {1048}}, //<PROD_ID2>
{51, {1018, 1040, 32, 21, 1041}}, //<EST_WRITE>
{52, {1038, 32, 21}}, //<EXPRESION>
{53, {-1}}, //<EXPRESION>
{54, {1019, 1040, 1000, 2002, 24, 1041}}, //<EST_READ>
{55, {1047, 1000}}, //<PREINCUNARIO>
{56, {1048, 1000}}, //<PREINCUNARIO>
{57, {1038, 1000, 2000, 24}}, //ID4
{58, {-1}}, //ID4
{59, {1022, 15, 1028, 1040, 32, 1041, 1032}}, //<EST_DO>
{60, {1020, 1040, 32, 1041, 15, 27, 28, 1031}}, //<EST_IF>
{61, {1025, 1040, 32, 1041, 15, 27}}, //<LISTA_ELSEIF>
{62, {-1}}, //<LISTA_ELSEIF>
{63, {1024, 15}}, //<OP_ELSE>
{64, {-1}}, //<OP_ELSE>
{65, {1021, 1040, 32, 1041, 15, 1029}}, //<EST_WHILE>
{66, {1023, 1000, 2002, 1040, 32, 1035, 32, 1041, 15, 1030}}, //<EST_FO
R>
{67, {1017, 32}}, //<EST_RETURN>
{68, {34, 2006, 33}}, //<EXPR>
{69, {1042, 2008, 34}}, //<EXPR'>
{70, {-1}}, //<EXPR'>
{71, {36, 2005, 35}}, //<EXPR2>
{72, {1043, 2007, 36}}, //<EXPR2'>
{73, {-1}}, //<EXPR2'>
{74, {37}}, //<EXPR3>
{75, {1046, 37}}, //<EXPR3>
{76, {39, 38}}, //<EXPR4>
{77, {43, 39}}, //<EXPR4'>
{78, {-1}}, //<EXPR4'>
{79, {41, 40}}, //<EXPR5>
{80, {2006, 1044, 2008, 39}}, //<EXPR5'>

```

```

{81, {2006, 1045, 2008, 39}}, //<EXPR5'>
{82, {-1}}, //<EXPR5'>
{83, {47, 42}}, //<TERM>
{84, {2005, 1055, 2007, 41}}, //<TERM'>
{85, {2005, 1057, 2007, 41}}, //<TERM'>
{86, {2005, 1058, 2007, 41}}, //<TERM'>
{87, {2005, 1056, 2007, 41}}, //<TERM'>
{88, {-1}}, //<TERM'>
{89, {1049}}, //<OPREL>
{90, {1050}}, //<OPREL>
{91, {1053}}, //<OPREL>
{92, {1051}}, //<OPREL>
{93, {1054}}, //<OPREL>
{94, {1052}}, //<OPREL>
{95, {1040, 45, 1041}}, //<LLAMADA_F>
{96, {-1}}, //<LLAMADA_F>
{97, {1000, 2002, 46}}, //<ID5>
{98, {1038, 1000, 2000, 46}}, //<ID5'>
{99, {-1}}, //<ID5'>
{100, {1000, 2002, 44}}, //<FACT>
{101, {10, 2003}}, //<FACT>
{102, {2009, 1040, 32, 1041, 2010}} //<FACT>
};
}

```

6. Integración con el Parser (Parser)

El analizador sintáctico (**Parser**) fue modificado para reconocer los marcadores de acción semántica. Cuando la cima de la pila (**tope**) es un número de acción (≥ 2000), en lugar de buscar una producción o esperar un terminal, llama a la función **ejecutarAccionSemantica**.

sources/parser_parser.cpp (Bloque de Acciones en analizarSintaxis):

```

bool Parser::analizarSintaxis(/*...*/) {
    // ... (try, while, obtener tope y token)
    // ... (if para No-Terminal, else if para Terminal)
}

```

```

// 4. Es una ACCION SEMANTICA
} else if (tope >= 2000 && tope <= 2012) { // Rango de acciones

    // Llama a la funcion que contiene el switch
    ejecutarAccionSemantica(tope);

    // Saca la accion de la pila (no consume token de entrada)
    pilaSintactica.pop();

} else {
    // ... (Error)
}
// ... (fin del while y try)
}

```

Muestra el bloque **else if** añadido al **while** principal del parser. Cuando **tope** es un número de acción, llama a **ejecutarAccionSemantica(tope)** y simplemente saca la acción de la pila, sin avanzar en la entrada de tokens.

Y antes de esos cambios también se implemento esto:

```

        // Acción 1: Guardar ID (Reglas 12 y 14)
        Produccion produccion = Productions::obtenerProduccionPorIndice(produccionIdx);
        if (!produccion.empty()) {
            pilaSintactica.pop();
            pushProduccion(produccion);
            continue;
        }
    }
    mostrarErrorSintactico(token, 600 + (tope - 1));
    return false;
//-----
} else if (tope >= 1000 && tope <= 1059) {
    // Es un TERMINAL
    if (tope == terminal) {
        qDebug() << "Salio: " << tope << terminal;
    }
}

```

```

        pilaSintactica.pop();
        tokenActual++; // Avanzamos al siguiente token
    } else {
        mostrarErrorSintactico(token, 700);
        return false;
    }
}

```

Este bloque se ejecuta cuando el símbolo en el tope de la pila es un **Terminal** un token específico esperado por la gramática (como **1002** para **def** o **1037** para **;**), identificado numéricamente del 1000 al 1059.

- **Verifica Coincidencia (Match):** Compara el terminal esperado en la pila (**tope**) con el token actual de la entrada (**terminal**).
- **Éxito (Match):** Si coinciden (**tope == terminal**), el código fuente sigue la estructura gramatical esperada. En este caso:
 1. Hace pop el terminal de la **pilaSintactica**, ya que ha sido consumido.
 2. Avanza al siguiente token incrementando **tokenActual**, moviendo el foco del parser al siguiente lexema.
- **Manejo de Error (Mismatch):** Si **tope** y **terminal** no coinciden, se encontró un token inesperado. Llama a **mostrarErrorSintactico** (con código de error 700) y detiene el análisis (**return false**).

sources/parser_parser.cpp (Función **ejecutarAccionSemantica**):

```

void Parser::ejecutarAccionSemantica(int numAccion) {
    // Obtenemos el token ANTERIOR, que es el relevante para la accion
    // (excepto para acciones de precedencia, que no miran el token)
    Token tokenPrevio = (tokenActual > 0) ?
        tokensAnalisis[tokenActual - 1] :
        Token{};

    // (A veces necesitamos el token actual, para acciones "pre-token")
    Token tokenSiguiente = (tokenActual < tokensAnalisis.size()) ?
        tokensAnalisis[tokenActual] :

```

```

Token{};

switch (numAccion) {
// --- Acciones de Declaracion
case 2000: // PUSH_ID_DECLARAR
    semantico.accionDeclararId(tokenPrevio.lexema);
    break;
case 2001: // ASIGNAR_TIPO
    semantico.accionAsignarTipo(semantico.getTipoFromToken(tokenPrevio.estado));
    break;

// --- Acciones de Expresion

case 2002: // VERIFICAR_ID_Y_PUSH_TIPO
    semantico.accionVerificarIdYPushTipo(tokenPrevio.lexema);
    break;

// (Accion 1)
case 2003: // PUSH_TIPO_CONSTANTE
    semantico.accionPushTipoConstante(tokenPrevio.estado);
    break;

// (Accion 2)
case 2004: // PUSH_OPERADOR (=)
    semantico.accionPushOperador(tokenPrevio.estado); // OP_ASIG (109)
    break;

// (Accion 3)
case 2005: // RESOLVER_PRECEDENCIA_ALTA (*, /)
    semantico.accionResolverOpsPrecedencia(1);
    break;

// (Accion 4)
case 2006: // RESOLVER_PRECEDENCIA_BAJA (+, -)
    semantico.accionResolverOpsPrecedencia(2);
    break;
}

```

```

        // (Accion 5)
    case 2007: // PUSH_OPERADOR (*, /, etc.)
        semantico.accionPushOperador(tokenPrevio.estado);
        break;

    // (Accion 6)
    case 2008: // PUSH_OPERADOR (+, -, ||)
        semantico.accionPushOperador(tokenPrevio.estado);
        break;

    // (Accion 7)
    case 2009: // PUSH_FONDO_FALSO
        semantico.accionPushFondoFalso();
        break;

    // (Accion 8)
    case 2010: // POP_FONDO_FALSO
        semantico.accionPopFondoFalso();
        break;

    // (Accion 9)
    case 2011: // RESOLVER_ASIGNACION
        semantico.accionResolverAsignacion();
        break;

    case 2012: // VERIFICAR_ID_Y_PUSH_TIPO (L-Value, para asignacion) Por
    ahora, hace lo mismo que 2002, pero es po si la logica cambia
        semantico.accionVerificarIdYPushTipo(tokenPrevio.lexema);
        break;
    }
}

```

Esta función actúa como un despachador (**switch**). Recibe el número de acción (**numAccion**) y llama a la función correspondiente en el objeto **semantico**, a menudo pasándole información del token previo (el que acaba de ser aceptado por el parser).

7. Reporte de Errores Semánticos

Los errores detectados en **SemanticAnalyzer** (duplicidad, tipo inválido, etc.) se añaden a una lista de strings (**listaErroresSemánticos**) gestionada por el **Parser**. Al finalizar el análisis, **MainWindow** recupera esta lista del **Parser** y muestra los errores en el área designada de la interfaz gráfica.

sources/semantic_analyzer.cpp (Reportando un Error):

```
void SemanticAnalyzer::resolverUnaOperacion() {
    // ... (sacar op, tipo1, tipo2)
    std::string tipoResultado = CuboSemantico::verificar(op, tipo1, tipo2);

    if (tipoResultado == T_ERROR) {
        std::string opStr = opTokenToString(op);
        std::string errorMsg = "ERROR SEMÁNTICO: Tipos incompatibles (" + t
ipo1 + " " + opStr + " " + tipo2 + ")";
        qDebug() << QString::fromStdString(errorMsg); // Log para consola
        if (pListaErrores) {
            pListaErrores->push_back(errorMsg); // Añadir a la lista del Parser
        }
    }
    // ...
}
```

Muestra cómo, al detectar un error (**tipoResultado == T_ERROR**), se construye un mensaje (**errorMsg**) y se añade a la lista referenciada por **pListaErrores**.

sources/mainwindow.cpp (Mostrando Errores en UI):

```
// ... (obtener tokens)
// ... (Ejecuta el analisis)
// Muestra los errores semanticos en la UI
for (const std::string& err : errores) {
    ui->textEditError->append(QString::fromStdString(err));
}

// Reporta el estado final
if (sintaxisCorrecta && errores.empty()) {
    ui->textEditSintaxis->setText("Análisis sintáctico y semántico correcto
```

```
s");
    } else if (sintaxisCorrecta && !errores.empty()) {
        ui->textEditSintaxis->setText("Análisis sintáctico correcto, pero con errores semánticos.");
    } else {
        ui->textEditSintaxis->setText("Errores sintácticos encontrados.");
    }
}
```

Después de llamar a **analizarSintaxis**, se obtiene la lista de errores semánticos usando **parser.getErroresSemanticos()** y se itera sobre ella para mostrar cada mensaje en el **textEditError** de la interfaz.

8. Imprimir pila de símbolos

Se usa para seguir la lógica de las pilas y detectar que se haga correctamente la parte semántica y poder detectar mas fácilmente los errores

sources/semantic_analyzer.cpp (imprimir pila):

```
/*
 * Funcion para imprimir el estado actual de la pila tipos
 * Recibe la pila para valor (copia) para no afectar la original
 */
void SemanticAnalyzer::debugImprimirPilaTipos(std::stack<std::string> pila) const {
    QDebug debug = qDebug().nospace().noquote();
    debug << " [TIPOS] (Top →): [";
    while (!pila.empty()) {
        debug << pila.top();
        pila.pop();
        if (!pila.empty()) debug << ", ";
    }
    debug << "];"
}
```

```
/*
 * Imprime el estado actual de la pila operadores
```

```

* recibe la pila por valor y traduce los tokens a strings
*/
void SemanticAnalyzer::debugImprimirPilaOperadores(std::stack<int> pila)
const {
    QDebug debug = qDebug().nospace().noquote();
    debug << " [OPS] (Top →): [";
    while (!pila.empty()) {
        int op = pila.top();
        if (op == FONDO_FALSO) {
            debug << "MFF"; // Marca de Fondo Falso
        } else {
            // Reutilizamos tu función 'opTokenToString' para ver '+', '*', etc.
            debug << opTokenToString(op);
        }
        pila.pop();
        if (!pila.empty()) debug << ", ";
    }
    debug << "];";
}

```

Implementacion de Cuadрупlos

Estructura del cuádruplo

Se crea un nuevo archivo `cuadрупlo.h`

```

#ifndef CUADRUPLO_H
#define CUADRUPLO_H

#include <string>
#include <iostream>

struct Cuadрупlo {
    std::string op; // Operador: +, -, *, /, =, SF, SI, GOTO, etc.
    std::string op1; // Operando 1
    std::string op2; // Operando 2 (puede estar vacío)
}

```

```

std::string res; // Resultado o Target del salto

// Constructor para facilitar la creación
Cuadruplo(std::string o, std::string o1, std::string o2, std::string r)
    : op(o), op1(o1), op2(o2), res(r) {}
};

#endif

```

Actualizamos SemanticAnalyzer (Variables y Pilas Nuevas)

Donde crearemos **Pila de Operandos (trabajando a la par con tipos, guarda nombres o los temporales), pila Saltos (accede por indice), y por ultimo la pila de cuadрупlos generados**

```

#include "cuadruplo.h"
class SemanticAnalyzer {
private:
std::stack<std::string> pilaOperandos; // Guarda "x", "5", "T1"
std::vector<int> pilaSaltos;          // Pila de saltos (vector para acceso random)
std::vector<Cuadruplo> listaCuadрупlos; // El código generado
int contadorTemporales;

public:
    // (funciones anteriores)

    // Nuevas funciones para Cuadрупlos
    void generarCuadruplo(std::string op, std::string op1, std::string op2, std::string res);
    void rellenar(int direccion, int valor); // Backpatching
    std::string nuevoTemporal();

    // Acciones para Estatutos (3000+)
    void accionIf_Inicio(); // 3000 (Después de EXPR)
    void accionIf_Else();   // 3001 (Antes de ELSE)
    void accionIf_Fin();    // 3002 (Al final de ENDIF)

```

```

void accionWhile_Inicio(); // 3003 (Antes de EXPR, guardar retorno)
void accionWhile_FinExpr(); // 3004 (Después de EXPR, generar SF)
void accionWhile_Fin(); // 3005 (Al final, generar SI y rellenar)

void accionDo_Inicio(); // 3006 (Guardar retorno)
void accionDo_Fin(); // 3007 (Generar SF al inicio si condición fall
a)

void accionFor_Inicio(); // 3008 (Asignación inicial)
void accionFor_Condicion(); // 3009 (Evaluar condicioon y SF)
void accionFor_Inc(); // 3010 (Incremento y SI) - Nota: Esto requiere
manejo especial en sintaxis
// Para el FOR simple del diagrama:
void accionFor_Fin(); // 3011

// Getters para ver el resultado
std::vector<Cuadрупло> getCuadрупlos() const { return listaCuadрупlos;
}
};

```

Implementar la Lógica (Generación y Saltos)

En semantic_analyzer.cpp se actualizo **resolverUnaOperacion** para que genere cuádruplos y agregar las funciones de saltos.

```

SemanticAnalyzer::SemanticAnalyzer() : cont_direcc(0), pListaErrores(nullptr),
contadorTemporales(1) {

}

void SemanticAnalyzer::reset() {
    // ... (lo anterior)
    // Limpiar lo nuevo
    while(!pilaOperandos.empty()) pilaOperandos.pop();
    pilaSaltos.clear();
    listaCuadрупlos.clear();
}

```

```

    contadorTemporales = 1;
}

```

Agregar las siguientes funciones auxiliares

```

void SemanticAnalyzer::generarCuadruplo(std::string op, std::string op1, std::string op2, std::string res) {
    listaCuadruplos.emplace_back(op, op1, op2, res);
    qDebug().noquote() << "GEN_QUAD: (" << op.c_str() << ", " << op1.c_str() << ", " << op2.c_str() << ", " << res.c_str() << ")";
}

std::string SemanticAnalyzer::nuevoTemporal() {
    return "R" + std::to_string(contadorTemporales++);
}

void SemanticAnalyzer::rellenar(int direccion, int valor) {
    if (direccion >= 0 && direccion < listaCuadruplos.size()) {
        listaCuadruplos[direccion].res = std::to_string(valor);
        qDebug().noquote() << "RELLENAR: Cuadruplo " << direccion << " con salto a " << valor;
    }
}

```

Actualizar **accionPush...** para **pilaOperandos**: Se actualizaron estas dos funciones para que, además de tipos, guarden los nombres en **pilaOperandos**

```

// (2002)
void SemanticAnalyzer::accionVerificarIdYPushTipo(const std::string& lexema) {
    //qDebug() << "Acción (2002): Verificando ID '" << QString::fromStdString(lexema) << "'";
    auto it = tablaSimbolos.find(lexema);
    if (it == tablaSimbolos.end()) {
        // 1. Mandar el error original
        std::string errorMsg = "ERROR SEMÁNTICO: Variable no declarada '"

```

```

+ lexema + "";
    qDebug().noquote() << errorMsg;
    if (pListaErrores) {
        pListaErrores→push_back(errorMsg);
    }
    // 2. se parchea con un real
    pilaTipos.push(T_FLOAT);
    qDebug().nospace().noquote() << "Push Tipo: [" << T_FLOAT << "];";
} else {
    qDebug().noquote() << "Acción (2002/12): Verificando ID '" << QString::fromStdString(lexema) << "'";
    pilaTipos.push(it→second.tipo);
    pilaOperandos.push(lexema); //para guardar el nombre
    historialOperandos.push_back(lexema); //para imprimir pilas
    qDebug().nospace().noquote() << "Push Tipo: [" << QString::fromStdString(it→second.tipo) << "];";
}
debugImprimirPilaTipos(pilaTipos);
}

```

```

// (Accion 1 → 2003)
void SemanticAnalyzer::accionPushTipoConstante(int tokenEstado, const std::string& lexema) {
    std::string tipo = getTipoConstanteFromToken(tokenEstado);
    qDebug().noquote() << "Acción (2003): Push Tipo Constante → " << QString::fromStdString(tipo);
    pilaTipos.push(tipo);
    pilaOperandos.push(lexema);
    historialOperandos.push_back(lexema); //imprimir pila
    debugImprimirPilaTipos(pilaTipos);
}

```

Actualizar **resolverUnaOperacion** (Generar Cuádruplos)

```

void SemanticAnalyzer::resolverUnaOperacion() {

    // 1.5 Sacar OPERANDOS (Nombres/Valores)

```

```

std::string op2_val = pilaOperandos.top(); pilaOperandos.pop();
std::string op1_val = pilaOperandos.top(); pilaOperandos.pop();

// 4. Generar Cuádruplo
if (op == OP_ASIG) { // Asignacion (=)
    // Cuádruplo: (=, op2, , op1) → Asigna op2 a op1
    generarCuadruplo("=", op2_val, "", op1_val);
    // En asignación no se genera temporal
    pilaOperandos.push(op1_val); // Dejamos el resultado por si acaso (a =
b = c)
} else {
    // Operación Aritmética/Lógica (+, -, *, <, >)
    std::string temporal = nuevoTemporal();
    generarCuadruplo(opStr, op1_val, op2_val, temporal);

    pilaOperandos.push(temporal); // Metemos el temporal T1 a la pila
    historialOperandos.push_back(temporal); // para cuádruplos en interfa
z
}
}

```

```

// (Accion 2, 5, 6 → 2004, 2007, 2008)
void SemanticAnalyzer::accionPushOperador(int opTokenEstado) {
    //qDebug().noquote() << "Acción (200x): Push Op → " << opTokenToStri
ng(opTokenEstado);
    pilaOperadores.push(opTokenEstado);
    historialOperadores.push_back(opTokenToString(opTokenEstado)); // par
a imprimir pila (guardamos simbolos)
    qDebug().nospace().noquote() << "Push Op: [" << opTokenToString(opT
okenEstado) << "]";
    debugImprimirPilaTipos(pilaTipos);
}

```

Acciones Semánticas para Estatutos

Aquí implementamos la lógica de saltos; En **semantic_analyzer.cpp**:

```
// --- IF
// Produccion: if ( EXPR ) <3000> ESTATUTOS <3001> else ESTATUTOS
endif <3002>

void SemanticAnalyzer::accionIf_Inicio() { // 3000
// 1. Generar SF (Salto Falso)
std::string cond = pilaOperandos.top(); pilaOperandos.pop();
pilaTipos.pop(); // Sacar el tipo bool

generarCuadruplo("SF", cond, "", "?"); // "?" es pendiente
pilaSaltos.push_back(listaCuadruplos.size() - 1); // Guardar indice del SF
}

void SemanticAnalyzer::accionIf_Else() { // 3001
// 1. Generar SI (Salto Incondicional) para saltar el ELSE
generarCuadruplo("SI", "", "", "?");

// 2. Rellenar el SF anterior (el del IF) para que caiga aqui (al inicio del else)
int falso = pilaSaltos.back(); pilaSaltos.pop_back();
rellenar(falso, listaCuadruplos.size()); // Apunta al siguiente cuadruplo actual

// 3. Guardar el indice del SI para rellenarlo al final
pilaSaltos.push_back(listaCuadruplos.size() - 1);
}

void SemanticAnalyzer::accionIf_Fin() { // 3002
// 1. Rellenar el SI (el que salta el else) o el SF (si no hubo else)
int fin = pilaSaltos.back(); pilaSaltos.pop_back();
rellenar(fin, listaCuadruplos.size());
}
```

```

// --- WHILE
// Produccion: while <3003> ( EXPR ) <3004> ESTATUTOS endwhile <300
5>

void SemanticAnalyzer::accionWhile_Inicio() { // 3003
    // 1. Guardar donde empieza la condicion para volver a evaluar
    pilaSaltos.push_back(listaCuadрупlos.size());
}

void SemanticAnalyzer::accionWhile_FinExpr() { // 3004
    // 1. Generar SF si la condicion falla
    std::string cond = pilaOperandos.top(); pilaOperandos.pop();
    pilaTipos.pop();

    generarCuadрупlo("SF", cond, "", "?");
    pilaSaltos.push_back(listaCuadрупlos.size() - 1); // Guardar indice SF
}

void SemanticAnalyzer::accionWhile_Fin() { // 3005
    int falso = pilaSaltos.back(); pilaSaltos.pop_back(); // Sacar SF
    int retorno = pilaSaltos.back(); pilaSaltos.pop_back(); // Sacar Retorno

    // 1. Generar SI para volver a evaluar
    generarCuadрупlo("SI", "", "", std::to_string(retorno));

    // 2. Rellenar el SF para que salga acá
    rellenar(falso, listaCuadрупlos.size());
}

```

```

// --- FOR
// Produccion: for id = EXPR to EXPR <3008> do ESTATUTOS endfor <30
11>

void SemanticAnalyzer::accionFor_Asignacion() { // 3008
    std::string vallnicial = pilaOperandos.top(); pilaOperandos.pop();

```

```

pilaTipos.pop();

// OJO: No sacamos el ID de la pila todavía, lo necesitaremos para la co
mparación
// y el incremento. Solo lo "leemos" (top) o lo dejamos ahí.
// Estrategia: Dejaremos el ID en la pilaOperandos durante TODO el ciclo.
std::string id = pilaOperandos.top();
// (No hacemos pop del ID ni de su tipo)

// Generar Cuádruplo: id = valInicial
generarCuádruplo("=", valInicial, "", id);
qDebug().noquote() << "Acción (3008): For - Asignación Inicial (" << QSt
ring::fromStdString(id) << "=" << QString::fromStdString(valInicial) << ")";
}

// Se llama inmediatamente después de la 3008
void SemanticAnalyzer::accionFor_Inicio() { // 3009
// Guardar el retorno (donde empezará la comparación en cada vuelta)
int direccionInicio = listaCuádruplos.size();
pilaSaltos.push_back(direccionInicio);
historialSaltos.push_back(listaCuádruplos.size() - 1);

/* Generar SF (Salirse si condición falsa)
generarCuádruplo("SF", cond, "", "?");
pilaSaltos.push_back(listaCuádruplos.size() - 1);*/
}

// Se llama después de: ... to EXP2 <AQUI> )
void SemanticAnalyzer::accionFor_Comparacion() { // 3010
// Pilas: [..., id, EXP2]
std::string valFinal = pilaOperandos.top(); pilaOperandos.pop();
pilaTipos.pop();

std::string id = pilaOperandos.top(); // El ID sigue ahí abajo

// 1. Generar Comparación: id < valFinal
std::string temporal = nuevoTemporal();
generarCuádruplo("<", id, valFinal, temporal);

```

```

// 2. Generar SF (Salto Falso) para salir si no se cumple
generarCuadriplo("SF", temporal, "", "?");

// 3. Guardar índice del SF para rellenar luego
pilaSaltos.push_back(listaCuadriplos.size() - 1);
historialSaltos.push_back(listaCuadriplos.size() - 1);

qDebug().noquote() << "Acción (3010): For - Comparación y SF";
}

void SemanticAnalyzer::accionFor_Fin() { // 3011
    // 1. Incrementar el ID: id = id + 1
        // Necesitamos un '1' constante.
        std::string id = pilaOperandos.top(); // Sigue ahí!
    std::string temporalInc = nuevoTemporal();

    // Generamos: T_inc = id + 1
    generarCuadriplo("+", id, "1", temporalInc);
    // Generamos: id = T_inc
    generarCuadriplo "=", temporalInc, "", id);

    // 2. Salto Incondicional (SI) al inicio (acción 3009)
    int falso = pilaSaltos.back(); pilaSaltos.pop_back(); // Sacamos el SF
    int retorno = pilaSaltos.back(); pilaSaltos.pop_back(); // Sacamos el Inicio

    generarCuadriplo("SI", "", "", std::to_string(retorno));

    // 3. Rellenar el SF para que apunte aquí (al final del loop)
    rellenar(falso, listaCuadriplos.size());

    // 4. ¡LIMPIEZA! Ahora sí sacamos el ID que guardamos todo este tiempo
    pilaOperandos.pop(); // Adiós id
    pilaTipos.pop(); // Adiós tipo id

    qDebug().noquote() << "Acción (3011): For - Incremento y Fin";
}

```

```

// --- DO-WHILE (REPEAT)

// Se llama justo después del token 'do' (1022)
void SemanticAnalyzer::accionDo_Inicio() { // 3006
    // Guardamos la direccion actual (el inicio del bucle) en la pila de saltos
    int direccionInicio = listaCuadрупlos.size();
    pilaSaltos.push_back(direccionInicio);
    qDebug().noquote() << "Acción (3006): Inicio Do-While en cuádruplo: "
    << direccionInicio;
}

// Se llama al final, después de 'dowhile ( EXPR )'
void SemanticAnalyzer::accionDo_Fin() { // 3007
    // 1. Sacar el resultado de la condicion (debe ser bool)
    std::string cond = pilaOperandos.top(); pilaOperandos.pop();
    pilaTipos.pop(); // Sacar el tipo (deberia ser BOOL, se puede validar)

    // 2. Sacar la dirección de retorno de la pila de saltos
    int retorno = pilaSaltos.back(); pilaSaltos.pop_back();

    // 3. Generar SV (Salto Verdadero)
    // Si la condicion es VERDADERA, volvemos al inicio (retorno)
    generarCuadрупlo("SV", cond, "", std::to_string(retorno));

    qDebug().noquote() << "Acción (3007): Fin Do-While (SV a " << retorno
    << ")";
}

```

La lógica es: Marcar el inicio → Ejecutar código → Evaluar condición → Si es Verdadera, saltar al inicio (SV).

- **Acción 3006 (accionDo_Inicio):** Guarda la dirección a donde debemos regresar.
- **Acción 3007 (accionDo_Fin):** Evalúa la condición y genera el salto.

Integrar en la Gramática (**parser_productions.cpp**)

```
// if ( EXPR ) <3004> ESTATUTOS <3000> LISTA_ELSEIF OP_ELSE endif <3002>
{60, {1020, 1040, 32, 1041, 3000, 15, 27, 28, 3002, 1031}}, //<EST_IF>

// EST_WHILE
// while <3003> ( EXPR ) <3004> ESTATUTOS endwhile <3005>
{65, {1021, 3003, 1040, 32, 1041, 3004, 15, 3005, 1029}}, //<EST_WHILE>

// EST_FOR
// for id <2002> ( EXPR <3008><3009> to EXPR <3010> ) ESTATUTOS en
dfor <3011>
// Prod 66 original: {1023, 1000, 1040, 32, 1035, 32, 1041, 15, 1030}
// Nota: se agrego 2002 para meter el ID a la pila desde el principio.
{66, {1023, 1000, 2002, 1040, 32, 3008, 3009, 1035, 32, 3010, 1041, 15, 1030, 3011}},

// EST_DO (REPEAT)
// do <3006> ESTATUTOS dowhile ( EXPR ) <3007> enddo
{59, {1022, 3006, 15, 1028, 1040, 32, 1041, 3007, 1032}}, //<EST_DO>

// EST_WRITE (Prod 51 original: {1018, 1040, 32, 21, 1041})
// write ( EXPR ) <3013>
{51, {1018, 1040, 32, 21, 1041, 3013}},

// EST_READ (Prod 54 original: {1019, 1040, 1000, 24, 1041})
// read ( id <2002> ) <3012>
{54, {1019, 1040, 1000, 2002, 24, 1041, 3012}},

//agregamos 2008 a perel
// Se agrega la acción 2008 (Push Operador) después de cada símbolo
{89, {1049, 2008}}, //<OPREL> ==
{90, {1050, 2008}}, //<OPREL> !=
{91, {1053, 2008}}, //<OPREL> <
{92, {1051, 2008}}, //<OPREL> <=
```

```
{93, {1054, 2008}}, //<OPREL> >
{94, {1052, 2008}}, //<OPREL> >=
```

Modificamos **accionResolverOpsPrecedencia**

```
void SemanticAnalyzer::accionResolverOpsPrecedencia(int precedencia) {
    qDebug().noquote() << "Acción (2005/2006): Checando Precedencia (Ni
vel " << precedencia << ")";
    while (!pilaOperadores.empty()) {
        int op = pilaOperadores.top();

        // 1. Definir Grupos de Operadores
        // Alta: *, /, %, **
        bool esPrecAlta = (op == OP_MULT || op == OP_DIV || op == OP_MOD
|| op == OP_POT);

        // Baja: +, -
        bool esPrecBaja = (op == OP_SUMA || op == OP_RESTA);

        // Relacional: >, <, >=, <=, ==, !=, &&, ||
        // (Los agregamos aquí para que se resuelvan cuando se pida precede
ncia baja o general)
        bool esRelacional = (op == OP_MAYOR || op == OP_MENOR || op == O
P_MAYOR_IGUAL ||
                            op == OP_MENOR_IGUAL || op == OP_IGUAL || op == OP
_DIFERENTE ||
                            op == OP_AND || op == OP_OR);

        // 2. Decidir si resolver
        if (precedencia == 1 && esPrecAlta) {
            resolverUnaOperacion();
        }
        // CAMBIO AQUÍ: Agregamos 'esRelacional' al chequeo de nivel 2
        else if (precedencia == 2 && (esPrecBaja || esRelacional)) {
            resolverUnaOperacion();
        } else {
            break; // No hay mas operaciones de esta precedencia
        }
    }
}
```

```

    }
}
}

```

Actualizar el switch en Parser

Ahora conectamos los numeros de las acciones creadas en **parcer_percaer.cpp**

```

void Parser::ejecutarAccionSemantica(int numAccion) {

    switch (numAccion) {
        // (2000 a 2012 existentes)

        // --- ESTATUTOS
        case 3000: semantico.accionIf_Inicio(); break;
        case 3001: semantico.accionIf_Else(); break; // Necesitarás poner este
en la producción de ELSE
        case 3002: semantico.accionIf_Fin(); break;

        case 3003: semantico.accionWhile_Inicio(); break;
        case 3004: semantico.accionWhile_FinExpr(); break;
        case 3005: semantico.accionWhile_Fin(); break;

        case 3006: semantico.accionDo_Inicio(); break;
        // y así sucesivamente
    }
}

```

generar cuádruplos para read y write (o print)

```

// --- READ (Leer) ---
// Producción: read ( id <2002> ) <3012>
// Nota: 2002 ya verificó que la variable existe.
void SemanticAnalyzer::accionRead() { // 3012
    // El ID está en la pila de operandos (lo metió la acción 2002)

```

```

std::string id = pilaOperandos.top(); pilaOperandos.pop();
pilaTipos.pop(); // Sacamos su tipo

// Generar: (READ, , , id)
generarCuadruplo("READ", "", "", id);
qDebug().noquote() << "Acción (3012): Generar READ para " << QString::fromStdString(id);
}

// --- WRITE (Escribir) ---
// Producción: write ( EXPR ) <3013>
void SemanticAnalyzer::accionWrite() { // 3013
    // El resultado de la expresión está en la pila
    std::string resultado = pilaOperandos.top(); pilaOperandos.pop();
    pilaTipos.pop();

    // Generar: (WRITE, , , result)
    generarCuadruplo("WRITE", "", "", resultado);
    qDebug().noquote() << "Acción (3013): Generar WRITE de " << QString::fromStdString(resultado);
}

```

En **parser_parser.cpp**

```

switch (numAccion) {
    // ... anteriores
    case 3012: semantico.accionRead(); break;
    case 3013: semantico.accionWrite(); break;
}

```

Formatos para la interfaz y imprimir pilas y cuádruplos

Necesitamos vectores adicionales para guardar el historial y funciones para obtenerlo y ponerlo en la interfaz

Agregamos a la funcion de reset

```
void SemanticAnalyzer::reset() {  
  
    // Limpiar historiales  
    historialOperandos.clear();  
    historialOperadores.clear();  
    historialSaltos.clear();  
}
```

```
// (Accion 7 → 2009)  
void SemanticAnalyzer::accionPushFondoFalso() {  
    //qDebug().noquote() << "Acción (2009): Push Op → MFF";  
    pilaOperadores.push(FONDO_FALSO);  
    historialOperadores.push_back("MFF"); //para la pila en interfaz  
    qDebug().nospace().noquote() << "Push Op: [MFF]";  
    debugImprimirPilaOperadores(pilaOperadores);  
}
```

En accionIf_Inicio, accionIf_Else, accionWhile_Inicio, accionFor_Inicio, agregar lo siguiente:

```
pilaSaltos.push_back(listaCuadрупlos.size() - 1);  
historialSaltos.push_back(listaCuadрупlos.size() - 1); //agregar esto
```

El MainWindow habla con el Parser, no directamente con el semantico. Necesitamos funciones "puente"

```
//agregar esto al final  
std::vector<std::string> Parser::getHistorialOperandos() const { return sem  
antico.getHistorialOperandos(); }  
std::vector<std::string> Parser::getHistorialOperadores() const { return sem  
antico.getHistorialOperadores(); }
```

```
std::vector<int> Parser::getHistorialSaltos() const { return semantico.getHistorialSaltos(); }
std::vector<Cuadruplo> Parser::getListaCuadruplos() const { return semantico.getListaCuadruplos(); }
```

Agregar lo siguiente para darle el formato en la interfaz

En void **MainWindow::on_btnSintactico_clicked()**

```
// -----
// 1. MOSTRAR HISTORIAL DE PILAS
// -----

// --- Pila de Operandos ---
QString sOperandos = "[";
std::vector<std::string> hOperandos = parser.getHistorialOperandos();
for (size_t i = 0; i < hOperandos.size(); ++i) {
    sOperandos += QString::fromStdString(hOperandos[i]);
    if (i < hOperandos.size() - 1) sOperandos += " | ";
}
sOperandos += "]";
ui->textEditPOperan->setText(sOperandos);

// --- Pila de Operadores ---
QString sOperadores = "[";
std::vector<std::string> hOperadores = parser.getHistorialOperadores();
for (size_t i = 0; i < hOperadores.size(); ++i) {
    sOperadores += QString::fromStdString(hOperadores[i]);
    if (i < hOperadores.size() - 1) sOperadores += " | ";
}
sOperadores += "]";
ui->textEditPOper->setText(sOperadores);

// --- Pila de Saltos ---
QString sSaltos = "[";
std::vector<int> hSaltos = parser.getHistorialSaltos();
for (size_t i = 0; i < hSaltos.size(); ++i) {
```

```

        sSaltos += QString::number(hSaltos[i]);
        if (i < hSaltos.size() - 1) sSaltos += " | ";
    }
    sSaltos += " ]";
    ui->textEditSaltos->setText(sSaltos);

// -----
// 2. MOSTRAR TABLA DE CUADRUPLOS
// -----
QString tablaCuadрупlos;

tablaCuadрупlos += QString("%1 | %2 | %3 | %4 | %5\n")
                    .arg("#", 3)
                    .arg("Oper", 6)
                    .arg("Op1", 10)
                    .arg("Op2", 10)
                    .arg("Res", 10);
tablaCuadрупlos += QString("-").repeated(50) + "\n";

std::vector<Cuadрупло> cuadрупlos = parser.getListCuadрупlos();

for (size_t i = 0; i < cuadрупlos.size(); ++i) {
    // Formateamos cada linea para que se vea alineada
    QString linea = QString("%1 | %2 | %3 | %4 | %5")
                    .arg(i, 3) // Numero del cuádruplo
                    .arg(QString::fromStdString(cuadрупlos[i].op), 6)
                    .arg(QString::fromStdString(cuadрупlos[i].op1), 10)
                    .arg(QString::fromStdString(cuadрупlos[i].op2), 10)
                    .arg(QString::fromStdString(cuadрупlos[i].res), 10);

    tablaCuadрупlos += linea + "\n";
}

// Verificar si la fuente es monoespaciada
QFont font("Courier New");
font.setStyleHint(QFont::Monospace);
ui->textEditCuafrupplos->setFont(font);
ui->textEditCuafrupplos->setText(tablaCuadрупlos);

```

Importante

Dentro del parser_parser.cpp dentro de donde se analiza el sintactico y donde se hace el salto para analizar la semantica; poner el rango a 3013 o 4000 para que se puedan analizar las nuevas acciones de los estatutos

```
else if (tope >= 2000 && tope <= 4000) { //ajustar el rango cando se agreg
uen acciones
    // ---- Es una ACCION SEMANTICA

    // 1. Llama a la funcion que tiene el SWITCH adentro
    ejecutarAccionSemantica(tope);

    // 2. Saca la accion de la pila.
    pilaSintactica.pop();

}
```

CONCLUSIÓN

El compilador Talos en el estado actual es capaz de analizar y validar semánticamente (con estatutos) código, reportando errores de forma clara, el cual también tiene una estructura sintáctica y semántica validada.